RAPPORT FINAL

Filière SMART-ICT / INE3 — Module Connectivity

# Allocation dynamique de ressources par IA pour le Network Slicing B5G


**RÉALISÉ PAR**

Fatima-Ezzahra GHANAM

Houda BOUYA

Filière SMART-ICT – INE3

**ENCADRÉ PAR**

Pr. Hassan EL GHAZI


**MODULE**

Connectivity

**ANNÉE UNIVERSITAIRE**

2026/2027



# Résumé

Dans ce projet, nous avons exploité le jeu de données **B5G Network Slicing** (165 simulations indépendantes, 37 312 flux) afin de répondre à une question centrale : peut-on apprendre, à partir des données, une prédiction utile pour la gestion des ressources des tranches de réseau (*slices*), et cette prédiction reste-t-elle fiable lors de changements brusques du réseau ?

Nous avons formulé le problème comme la **prédiction du logarithme du délai moyen de chaque flux**, à partir des seules entrées de la simulation (trafic offert, type de tranche, chemin routé, capacités, configuration de l'ordonnancement). Nous avons suivi les étapes demandées par le guide : exploration des données, formulation du problème, modèle de référence, définition des événements, représentation sous forme de graphe, réseau de neurones sur graphes, puis comparaison, ablations et analyse, en séparant systématiquement les données **par simulation** et en testant deux protocoles : un protocole « random » (même distribution) et un protocole « shift » (test sur les réseaux les plus grands).

Nos résultats montrent que la congestion du goulot d'étranglement et la configuration de l'ordonnancement déterminent la qualité de service. Le modèle d'arbres de gradient (HGB) est le plus précis (erreur quadratique moyenne de 2,12 en protocole « random » et de 3,37 en protocole « shift »). Le réseau de neurones sur graphes (GNN) apprend environ 60 % de l'information de congestion sans qu'aucune caractéristique de charge ne lui soit fournie, mais il se dégrade fortement lorsque la taille du réseau change. Nous concluons que le GNN apporte une valeur ajoutée surtout lorsque la contention n'est pas décrite par des caractéristiques ingéniérées à la main, et à condition que la distribution des données reste proche de celle de l'entraînement.

**Mots-clés :** network slicing, B5G, qualité de service, délai, WFQ, réseaux de neurones sur graphes, GATv2, GraphSAGE, robustesse, changement de distribution.

# Sommaire

1. Introduction et cadre du projet
2. Organisation du travail et du code
3. Jeu de données et pièges identifiés
4. L1 — Exploration des données et description des variables
5. L2 — Problématique : cible, hypothèses et protocole expérimental
6. L3 — Modèle de référence et modèle d'apprentissage automatique
7. L4 — Détection et analyse des événements
8. L5 — Représentation en graphe et réseau de neurones sur graphes
9. L6 — Comparaison, ablations, robustesse et analyse
10. Limites de notre approche
11. Réponse à la question finale et conclusion

Annexe — Reproductibilité des résultats

## Liste des abréviations utilisées

***Tableau 1 –*** *Abréviations employées dans ce rapport*

| Abréviation | Signification |
|---|---|
| **B5G** | Beyond 5G : réseaux au-delà de la cinquième génération |
| **QoS** | Quality of Service : qualité de service (ici, le délai et les pertes) |
| **SLA** | Service Level Agreement : accord de niveau de service (ici, le délai maximal toléré δ de la tranche) |
| **eMBB** | Enhanced Mobile Broadband : haut débit mobile amélioré |
| **URLLC** | Ultra-Reliable Low-Latency Communications : communications ultra-fiables à faible latence |
| **mMTC** | Massive Machine-Type Communications : communications massives entre machines |
| **WFQ / FIFO** | Weighted Fair Queuing (mise en file pondérée équitable) / First In, First Out (premier arrivé, premier servi) |
| **ToS** | Type of Service : champ qui associe un paquet à une file d'attente |
| **MAE / MSE / RMSE** | Erreur absolue moyenne / erreur quadratique moyenne / racine de l'erreur quadratique moyenne |
| **$R^2$** | Coefficient de détermination : part de la variance expliquée par le modèle |
| **MAD** | Median Absolute Deviation : déviation absolue médiane (mesure de dispersion robuste) |
| **HGB** | HistGradientBoosting : arbres de décision à gradient boosté |
| **MLP** | Multi-Layer Perceptron : perceptron multicouche (réseau de neurones sans graphe) |
| **GNN** | Graph Neural Network : réseau de neurones sur graphes |
| **GCN / SAGE / GATv2** | Graph Convolutional Network / GraphSAGE / Graph Attention Network version 2 : trois familles de couches de GNN |
| **ρ (rho)** | Charge normalisée : débit offert divisé par la capacité |

# 1. Introduction et cadre du projet

## 1.1 Contexte : le découpage en tranches (network slicing)

Les réseaux de nouvelle génération (5G puis au-delà de la 5G, noté B5G) reposent sur le **découpage en tranches** du réseau : une même infrastructure physique est partagée entre plusieurs tranches virtuelles, chacune adaptée à un type de service. Trois familles sont étudiées dans ce projet : la tranche **eMBB** (haut débit, exigence principale : le débit), la tranche **URLLC** (exigence principale : une latence très faible et une grande fiabilité) et la tranche **mMTC** (exigence principale : la massivité, c'est-à-dire un très grand nombre d'objets connectés à faible débit).

Ces tranches se disputent des ressources communes : la capacité des liens et la place dans les files d'attente des routeurs. Dans notre jeu de données, l'ordonnancement des paquets se fait soit par **WFQ** (chaque file reçoit un poids, donc une part garantie de la capacité), soit par **FIFO**. Pour gérer dynamiquement ces ressources, un opérateur doit pouvoir **prévoir la qualité de service avant de déployer une allocation** : le délai exigé par la tranche sera-t-il respecté, et quelle tranche sera dégradée ? C'est le cœur de notre étude.

## 1.2 Mission, question centrale et question finale

Le guide du projet nous demande d'exploiter le jeu de données B5G Network Slicing pour étudier la gestion des ressources des tranches et pour évaluer la robustesse d'une approche d'apprentissage lors de changements brusques du réseau. Deux questions structurent notre travail.

> **Question centrale.** « Peut-on apprendre à partir des données une stratégie ou une prédiction utile pour la gestion des ressources des slices, et cette solution reste-t-elle fiable lors de changements brusques du réseau ? »
>
> **Question finale.** « Dans quelles conditions une approche GNN apporte-t-elle une valeur ajoutée pour la gestion dynamique des slices B5G ? »

Le guide précise que nous devons **formuler nous-mêmes le problème**, construire une démarche expérimentale argumentée (et non reproduire une solution fournie) et proposer une interprétation : un meilleur score ne signifie pas nécessairement une meilleure solution. Nous avons gardé cette consigne comme fil conducteur : chaque choix de ce rapport est justifié, et chaque résultat est accompagné de ses limites.

## 1.3 Notre démarche en huit étapes

Nous avons suivi les huit étapes du guide, dans l'ordre. La figure ci-dessous résume l'enchaînement, et le tableau qui la suit indique, pour chaque étape, ce qui est demandé, le livrable correspondant (L1 à L6) et l'endroit de ce rapport où il est traité. Le livrable L7 correspond à la soutenance : nous l'avons préparée en défendant trois choix méthodologiques (section 11.4).

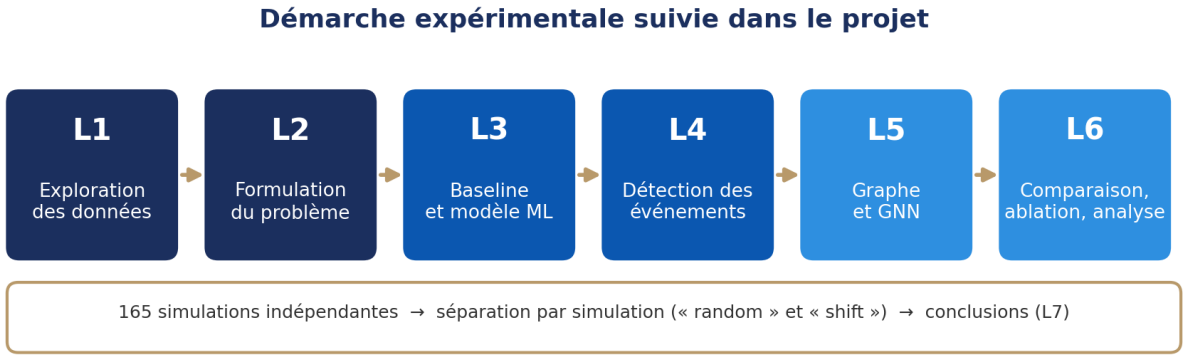



***Figure 1 –*** *Enchaînement des étapes de notre démarche expérimentale (L1 à L6, puis préparation de la soutenance L7).*

*Tableau 2 – Les huit étapes du guide, les livrables et leur traitement dans notre rapport*

| Étape du guide | Ce qui est demandé | Livrable | Section |
|---|---|---|---|
| **1. Explorer** | Fichiers, variables, types, valeurs manquantes, doublons, anomalies, distributions ; rôle de chaque variable (demande, ressource, qualité de service) | L1 | 4 |
| **2. Formuler le problème** | Choisir la cible (demande, qualité de service, allocation, classe…) et défendre pourquoi c'est un problème de gestion de slices | L2 | 5 |
| **3. Modèle de référence** | Un modèle simple, séparation entraînement / validation / test sans fuite d'information, MAE et MSE au minimum ; aucun GNN sans comparaison à la référence | L3 | 6 |
| **4. Événement** | Mesure de changement, règle ou seuil justifié, masque événement / non-événement, vérification visuelle | L4 | 7 |
| **5. Graphe** | Nœuds, arêtes, caractéristiques, graphe homogène ou hétérogène, pourquoi un graphe est plus informatif qu'un vecteur | L5 | 8.1 – 8.2, 8.6 |
| **6. GNN** | GCN, GraphSAGE ou GAT : choix justifié, entrées et sorties, agrégation, hyperparamètres, contrôle du surapprentissage | L5 | 8.3 – 8.6 |
| **7. Comparer** | Référence, modèle d'apprentissage automatique et GNN en MAE et MSE ; évaluation globale et sur les événements ; ablation | L6 | 9.1 – 9.3 |
| **8. Analyser** | Qui généralise, qui résiste aux événements, gain réel du GNN, caractéristiques importantes, changement de distribution, limites | L6 | 9.4 – 11 |

# 2. Organisation du travail et du code

Nous avons structuré le projet en scripts Python indépendants, un par étape, afin que chaque résultat de ce rapport puisse être régénéré à partir des données brutes. Les outils utilisés sont Python 3.10, scikit-learn (modèles classiques) et PyTorch Geometric (réseaux de neurones sur graphes). L'interface de lecture des simulations, fournie avec le jeu de données, n'a pas été modifiée.

***Tableau 3 –*** *Architecture du code fourni avec le projet*

| Fichier | Rôle |
|---|---|
| **datanetAPI.py** | Interface fournie (non modifiée) : lit les topologies, les routages, les tranches et les résultats des simulations. |
| **01_extract.py** | Extrait trois tables (échantillons, flux, liens), les ressources (lien, file WFQ) et les structures nécessaires au graphe. |
| **features.py** | Construit les caractéristiques (entrées uniquement), la cible ln(délai) et le masque d'événements ; définit les groupes de caractéristiques pour l'ablation. |
| **02_eda.py / 03_events.py** | Étape L1 (exploration des données) et étape L4 (événements). |
| **04_baseline.py** | Étape L3 : moyenne, régression Ridge, HistGradientBoosting et ablation par groupe de caractéristiques. |
| **05_gnn.py** | Étape L5 : graphe flux–file–lien, GCN / GraphSAGE / GATv2, ablations du GNN, courbes d'apprentissage. |
| **06_compare.py, evaluation.py, utils.py** | Étape L6 : comparaison, robustesse, erreurs par groupe, importance des caractéristiques ; métriques et découpage par simulation. |

# 3. Jeu de données et pièges identifiés

Le jeu de données contient **165 simulations indépendantes** réparties dans 33 archives compressées. Chaque simulation possède sa propre topologie (de 26 à 699 nœuds), son propre jeu de tranches et sa propre matrice de trafic. L'ensemble représente **37 312 flux** et **82 914 liens dirigés**. Les nœuds appliquent une politique d'ordonnancement WFQ ou FIFO, avec des poids WFQ définis par port de sortie et une correspondance entre le champ ToS et la file d'attente.

En explorant les fichiers, nous avons découvert trois pièges qu'il faut connaître pour interpréter correctement les résultats. Chacun a une conséquence directe sur nos choix méthodologiques.

***Tableau 4 –*** *Les trois pièges du jeu de données et leurs conséquences*

| Piège identifié | Explication | Conséquence sur notre démarche |
|---|---|---|
| **Pas de série temporelle** | Les simulations sont indépendantes et l'intensité de trafic est unique ($18{\cdot}10^6$). Un « changement brusque » ne peut donc être qu'un changement de régime d'un flux (fonctionnement normal → congestion sévère) ou un changement de distribution entre simulations (taille du réseau). | Nous définissons les événements comme des basculements de régime (section 7) et le changement de distribution comme un test sur de plus grands réseaux (protocole « shift »). |
| **Chemin planifié ≠ chemin simulé** | Le chemin décrit dans les fichiers de tranches et les attributs de trafic et de réservation des liens sont un plan. Le chemin réellement simulé est celui du fichier de routage : sa corrélation avec l'utilisation réelle des liens est de 0,93 à 0,98, contre 0,4 à 0,7 pour le plan. | Nous construisons les charges à partir du **routage** et traitons les caractéristiques « planifiées » avec prudence. |
| **Sorties du simulateur** | Le délai moyen, le nombre de paquets perdus et l'utilisation des liens sont des résultats de simulation. | Nous ne les utilisons **jamais** comme entrées, afin d'éviter toute fuite d'information. |

# 4. L1 — Exploration des données et description des variables

Pour cette première étape, nous avons inventorié les fichiers et les variables, vérifié leur qualité, puis étudié leurs distributions et leurs relations. Pour chaque variable, nous avons indiqué sa signification, son type, son rôle potentiel (demande, ressource ou qualité de service) et la justification de ce rôle, comme le demande le guide.

## 4.1 Tableau des variables

***Tableau 5 –*** *Variables du jeu de données : signification, type, rôle et justification*

| Variable | Signification | Type | Rôle | Justification |
|---|---|---|---|---|
| **sample_id** | Identifiant d'une simulation (une topologie, un jeu de tranches, un trafic) | identifiant | Regroupement | Unité de séparation entraînement / validation / test : les flux d'une simulation partagent les mêmes liens, ce qui évite la fuite d'information. |
| **N, n_links, n_flows** | Nombre de nœuds, de liens dirigés et de flux actifs | entier | Contexte | Mesure la taille du réseau ; sert à créer le changement de distribution (protocole « shift »). |
| **slice_type** | Type de tranche : eMBB, URLLC ou mMTC | catégorielle | DEMANDE (type de service) | Les trois tranches ont des exigences différentes (débit, latence, massivité). |
| **delta** | Exigence de délai de la tranche (SLA) | continu | Cible de qualité de service / SLA | Permet de définir une violation de SLA. |
| **bw (EqLambda)** | Débit offert du flux : 0,2 / 3,5 / 18 Mbit/s | continu discret | DEMANDE | Charge injectée dans le réseau. |
| **pkt_size, pkts_lambda** | Taille moyenne des paquets (bits) et débit de paquets | continu | DEMANDE | Détermine le temps de service et la congestion. |
| **hops** | Nombre de liens du chemin routé | entier | RESSOURCE (chemin) | Plus de sauts signifie plus de files d'attente traversées. |
| **capacity** | Capacité du lien (bit/s) : 50 / 100 / 350 / 400 Mbit/s | continu | RESSOURCE | Bande passante disponible. |
| **is_wfq, queue_size** | Politique d'ordonnancement du nœud émetteur et taille des files | catégorielle / entier | RESSOURCE (configuration) | Partage de la capacité entre tranches (WFQ) ou file unique (FIFO). |
| **n_flows_link, load, rho** | Nombre de flux, charge offerte et charge offerte divisée par la capacité (d'après le routage) | continu | DEMANDE comparée à la RESSOURCE | Indicateur direct de congestion, calculé uniquement à partir des entrées. |
| **planned_reservation, planned_traffic** | Réservation et trafic planifiés par tranche sur le lien (chemins de tranches, non simulés) | continu | RESSOURCE allouée | Partiellement incohérents avec le routage simulé (corrélation 0,4 à 0,7) : à utiliser avec prudence. |
| **avg_delay (cible)** | Délai moyen du flux (s), de $4{\cdot}10^{-6}$ à 1825 | continu, très asymétrique | QoS (SORTIE) | Cible : nous prédisons ln(délai). |
| **loss_ratio** | Pertes divisées par les paquets générés | continu dans [0, 1] | QoS (SORTIE) | Utilisé pour les événements et l'analyse, jamais comme entrée. |
| **jitter, p90** | Gigue et 90e percentile du délai | continu | QoS (SORTIE) | Métriques alternatives de qualité de service. |
| **util_sim, loss_sim** | Utilisation et pertes simulées des liens | continu | QoS réseau (SORTIE) | Analyse et visualisation des événements ; jamais en entrée (fuite d'information). |

## 4.2 Qualité des données

*Tableau 6 – Contrôles de qualité des données*

| Contrôle effectué | Valeur obtenue |
|---|---|
| Simulations (échantillons) | 165 |
| Flux | 37 312 |
| Liens dirigés | 82 914 |
| Valeurs manquantes (flux) | 0 |
| Valeurs manquantes (utilisation des liens) | 0 |
| Doublons (flux) | 0 |
| Délais inférieurs ou égaux à 0 | 0 |
| Taux de perte hors de l'intervalle [0, 1] | 0 |
| Intensité de trafic (valeurs distinctes) | 1 seule ($18 \cdot 10^6$) |
| Liens d'utilisation légèrement supérieure à 1 (arrondi du simulateur) | 92 |

**Lecture.** Les données sont propres : nous n'avons trouvé ni valeur manquante, ni doublon, ni délai non positif, ni taux de perte hors de l'intervalle [0, 1]. Les 92 liens dont l'utilisation dépasse légèrement 1 résultent d'un effet d'arrondi ou de mesure du simulateur et n'ont aucun impact sur notre étude. Comme une seule intensité de trafic existe, la variabilité provient uniquement de la topologie, du routage, des tranches et de la configuration WFQ.

## 4.3 Distributions et relations

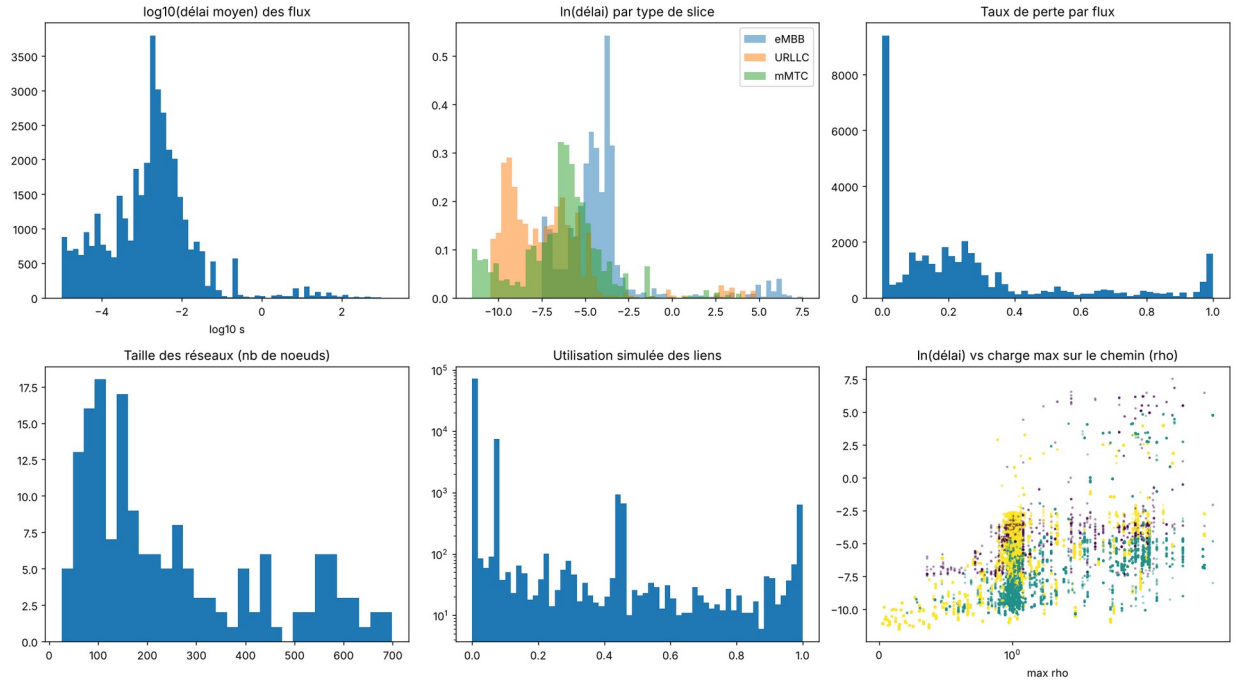



*Figure 2 – Distributions : délai (log10), ln(délai) par type de tranche, taux de perte, taille des réseaux, utilisation simulée des liens et ln(délai) en fonction de la charge maximale ρ sur le chemin.*

**Analyse.** (i) Le délai moyen s'étale de 4 µs à 1825 s : la transformation logarithmique est indispensable, elle donne une distribution quasi symétrique avec une longue queue à droite. (ii) Les tranches ont des niveaux de délai distincts (URLLC le plus bas, eMBB le plus haut) mais avec un fort recouvrement : le type de tranche seul ne suffit donc pas à prédire le délai. (iii) Les taux de perte sont bimodaux : environ 9 400 flux n'ont aucune perte, une masse de flux a des pertes modérées (0,1 à 0,4) et un pic apparaît à 1, qui correspond à l'effondrement du flux. (iv) La relation entre ln(délai) et ρ montre un **changement de régime autour de ρ ≈ 1** : sous la saturation le délai est faible, au-delà il explose, ce qui est la signature d'une relation non linéaire.

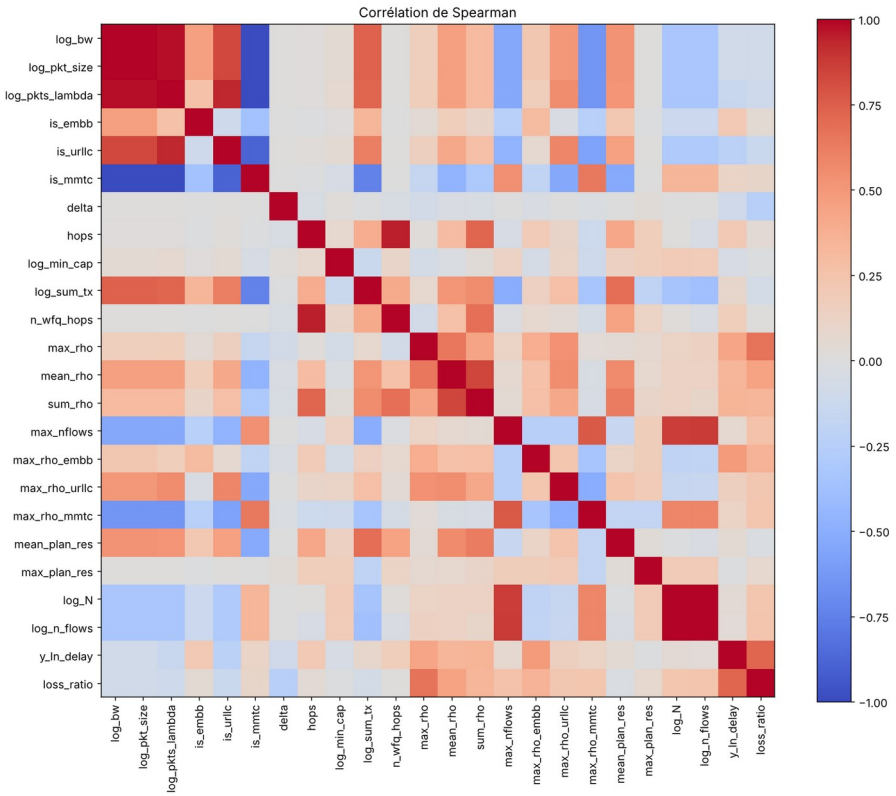



***Figure 3 –*** *Corrélations de Spearman entre les caractéristiques et les deux cibles (ln(délai) et taux de perte).*

***Tableau 7 –*** *Corrélations les plus fortes avec le délai et les pertes*

| Caractéristique | \|ρ de Spearman\| avec ln(délai) | \|ρ\| avec le taux de perte |
|---|---|---|
| **max_rho_embb** | 0,49 | 0,36 |
| **max_rho** | 0,44 | 0,67 |
| **sum_rho** | 0,35 | 0,34 |
| **mean_rho** | 0,34 | 0,45 |
| **is_urllc** | 0,22 | 0,13 |
| **hops** | 0,21 | 0,05 |
| **is_embb** | 0,21 | 0,05 |
| **n_wfq_hops** | 0,17 | 0,00 |

**Analyse.** Les caractéristiques de **charge** (ρ maximal sur le chemin, notamment la charge due à la tranche eMBB) sont les plus corrélées au délai (0,44 à 0,49) et aux pertes (max_rho : 0,67). Les caractéristiques du flux pris isolément (débit, taille de paquet, nombre de sauts) sont faiblement corrélées (inférieures à 0,21) : la qualité de service dépend surtout du *contexte de contention* et non du flux seul. Notons aussi que `log_bw` et `log_pkt_size` ont des corrélations identiques : les trois profils de trafic (0,2 / 3,5 / 18 Mbit/s) lient le débit et la taille de paquet.

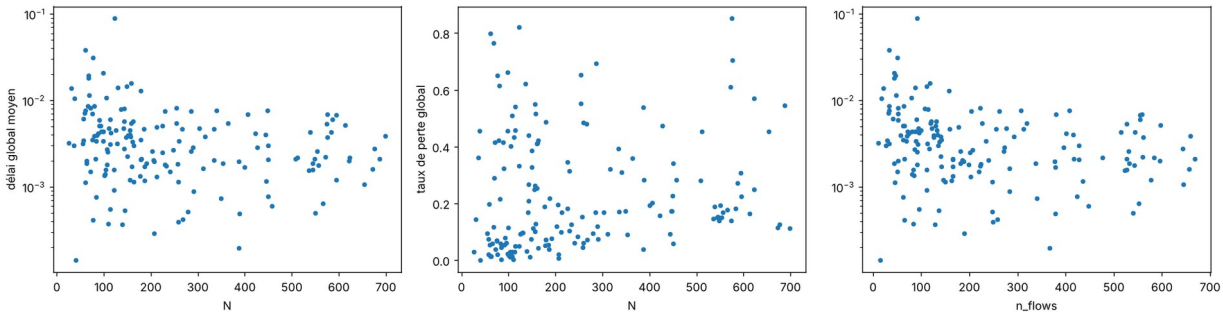



***Figure 4 –*** *Niveau simulation : délai global et taux de perte global en fonction de la taille du réseau (N) et du nombre de flux.*

Les simulations sont très hétérogènes : le taux de perte global varie de 0 à 0,85, sans tendance monotone avec N. Un modèle doit donc **généraliser entre topologies**, et pas seulement entre flux d'une même simulation. Cette observation justifie directement notre choix de séparer les données par simulation (section 5.3).

# 5. L2 — Problématique : cible, hypothèses et protocole expérimental

## 5.1 Formulation du problème

L'unité d'analyse est un **flux** (source → destination) appartenant à une tranche, dans une simulation donnée. La cible à prédire est le **logarithme népérien du délai moyen du flux**, noté ln(délai). Les entrées sont le trafic offert, le type de tranche et son exigence de délai δ, le chemin routé, les capacités, la configuration WFQ (part garantie par file) et la charge offerte. Aucune sortie du simulateur n'est utilisée comme entrée.

**Question du guide : Quelle est la cible choisie, et pourquoi est-ce un problème de gestion de slices ?**

Nous avons choisi comme cible le logarithme népérien du délai moyen de chaque flux, car le délai est la mesure de qualité de service qui traduit directement l'exigence de latence des tranches. Un opérateur doit savoir, avant de déployer une allocation (un chemin et des poids WFQ), si le délai exigé par la tranche sera respecté et quelle tranche sera dégradée. Prédire le délai à partir des seules entrées revient à construire un modèle de qualité de service, c'est-à-dire un « jumeau numérique » léger, qui permet d'évaluer une allocation candidate sans relancer une simulation coûteuse.

Ce modèle est ainsi la brique prédictive d'une allocation dynamique : il suffit de retenir l'allocation dont le délai prédit respecte l'exigence δ. Le passage au logarithme est justifié par l'échelle de variation du délai, qui couvre six ordres de grandeur ; il rend l'erreur relative et stabilise la perte quadratique.

## 5.2 Hypothèses testées

*Tableau 8 – Hypothèses de travail formulées avant les expériences*

| N° | Hypothèse | Vérifiée dans |
|---|---|---|
| **H1** | Le délai dépend surtout de la congestion (charge comparée à la capacité garantie sur le chemin) et non des seules caractéristiques du flux : un modèle sans caractéristiques de charge sera mauvais. | Sections 6 et 9.3 |
| **H2** | Les relations étant non linéaires (saturation autour de ρ ≈ 1), un modèle non linéaire bat un modèle linéaire. | Section 6 |
| **H3** | Un GNN, qui agrège la contention par les ressources partagées, bat un modèle sans structure (MLP) utilisant les mêmes caractéristiques individuelles. | Section 9.3 |
| **H4** | Sous changement de distribution (réseaux plus grands), les performances se dégradent ; les événements sont plus difficiles à prédire que les cas normaux. | Sections 9.2 et 9.4 |

## 5.3 Protocole expérimental

Nous avons défini le protocole suivant pour garantir une évaluation honnête :

- **Séparation par simulation, jamais par flux.** Les flux d'une simulation partagent les mêmes liens et les mêmes files : séparer par flux ferait fuir l'information de contention entre l'entraînement et le test.
- **Protocole « random » (même distribution).** 20 % des simulations sont réservées au test (33 simulations, soit 9 489 flux), 15 % à la validation (24 simulations) et le reste à l'entraînement (108 simulations).
- **Protocole « shift » (changement brusque de distribution).** L'entraînement et la validation portent sur les petits réseaux (N inférieur au quantile 65 %), et le test sur les 35 % de réseaux les plus grands : 25 619 flux de test contre seulement 11 693 en entraînement et validation, car les grands réseaux contiennent beaucoup plus de flux.

- **Sélection sur la validation uniquement.** Les hyperparamètres et l'architecture sont choisis sur la validation ; le test n'est jamais utilisé pour choisir.
- **Reproductibilité.** Graine fixée à 42 ; trois graines pour le GNN retenu afin de mesurer la variabilité.

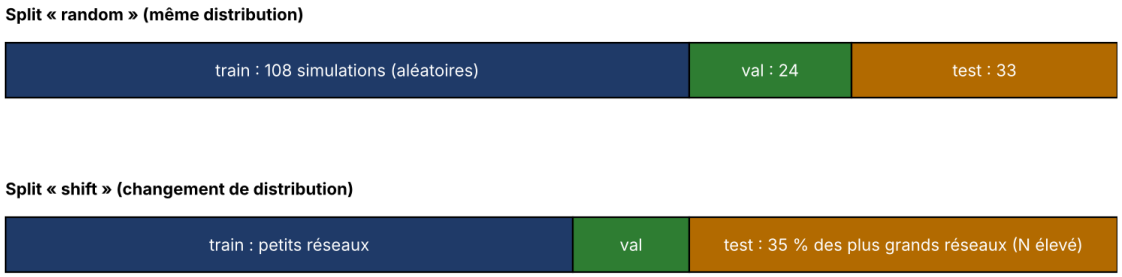



***Figure 5 –*** *Protocoles de séparation par simulation : « random » (même distribution) et « shift » (test sur les plus grands réseaux).*

## 5.4 Métriques d'évaluation

Nous évaluons chaque modèle sur ln(délai) avec quatre métriques : l'**erreur absolue moyenne (MAE)**, moyenne des écarts en valeur absolue entre valeur prédite et valeur réelle ; l'**erreur quadratique moyenne (MSE)**, moyenne des carrés des écarts, qui pénalise fortement les grosses erreurs ; sa racine, la **RMSE**, exprimée dans la même unité que la cible ; et le **coefficient de détermination $R^2$**, qui mesure la part de variance expliquée (1 pour une prédiction parfaite, 0 pour une prédiction égale à la moyenne, négatif si le modèle fait pire que la moyenne). Chaque métrique est calculée sur tous les flux, uniquement sur les événements et hors événements.

# 6. L3 — Modèle de référence et modèle d'apprentissage automatique

## 6.1 Modèles comparés et justification

Le guide impose un modèle simple de référence avant tout GNN. Nous avons construit trois modèles de complexité croissante, chacun répondant à une question précise : la moyenne mesure ce qu'on obtient sans aucune information ; la régression Ridge mesure ce que peut faire un modèle linéaire sans information de congestion ; HistGradientBoosting mesure ce que permet un modèle non linéaire disposant d'une description explicite de la congestion.

***Tableau 9 –*** *Les trois modèles de la phase de référence*

| Modèle | Caractéristiques utilisées | Rôle dans l'étude |
|---|---|---|
| **mean_train** | Aucune | Référence triviale : prédit la moyenne de ln(délai) sur l'entraînement. |
| **ridge_base** | Trafic, tranche, structure du chemin (nombre de sauts, capacité minimale, temps de transmission cumulé, nombre de sauts WFQ) | Référence linéaire **sans information de congestion** ; le coefficient de régularisation α, pris entre 0,01 et 100, est choisi sur la validation. |
| **hgb_all** | Toutes les caractéristiques : celles de ridge_base, plus la charge ρ sur le chemin (globale et par tranche), ρ par file WFQ, part WFQ minimale, rapport débit garanti / débit demandé, réservation planifiée, contexte (N) | Modèle d'apprentissage automatique non linéaire (HistGradientBoosting) ; trois configurations (taux d'apprentissage, profondeur) comparées sur la validation, régularisation L2 activée. |

**Caractéristiques de charge construites sans fuite d'information.** À partir du routage et du trafic offert uniquement, nous avons calculé : la charge d'un lien ρ = (somme des débits offerts des flux routés sur ce lien) / capacité du lien ; la charge d'une file WFQ = charge de la file / (part WFQ × capacité) ; et le rapport entre le débit garanti minimal et le débit du flux. Ces quantités ne dépendent que des entrées : elles décrivent la congestion *attendue* et non la congestion *observée*.

## 6.2 Résultats

***Tableau 10 –*** *Protocole « random » : résultats sur le jeu de test*

| Modèle | MAE événements | MAE globale | MSE événements | MSE globale |
|---|---|---|---|---|
| **mean_train** | 4,027 | 1,730 | 22,687 | 5,292 |
| **ridge_base** | 4,310 | 1,643 | 24,680 | 4,930 |
| **hgb_all** | **1,333** | **0,967** | **5,658** | **2,119** |

***Tableau 11 –*** *Protocole « shift » : résultats sur le jeu de test*

| Modèle | MAE événements | MAE globale | MSE événements | MSE globale |
|---|---|---|---|---|
| **mean_train** | 4,053 | 1,541 | 22,325 | 4,789 |
| **ridge_base** | 4,327 | 1,466 | 24,243 | 4,302 |
| **hgb_all** | **1,264** | **1,390** | **2,798** | **3,374** |

## 6.3 Analyse

> **H1 confirmée.** La régression Ridge n'améliore presque pas la moyenne (MSE globale de 4,93 contre 5,29) et elle est même moins bonne que la moyenne sur les événements (24,7 contre 22,7). Sans information de congestion, elle prédit des délais « normaux » y compris pour des flux saturés. La congestion est donc bien l'information décisive.
>
> **H2 confirmée.** HistGradientBoosting réduit la MSE globale de 57 % par rapport à Ridge (2,12 contre 4,93, soit un $R^2$ d'environ 0,59) et de 77 % sur les événements (5,66 contre 24,7). Un modèle non linéaire capte bien la rupture de régime autour de $\rho \approx 1$.

Sous changement de distribution, l'avance de HistGradientBoosting persiste (MSE de 3,37 contre 4,30 pour Ridge) mais l'écart se réduit : le $R^2$ chute de 0,59 à 0,29. Nous retenons donc **ridge_base comme référence simple** et **hgb_all comme référence forte** pour toutes les comparaisons avec le GNN.

# 7. L4 — Détection et analyse des événements

## 7.1 Définition et justification

> **Définition.** Un **événement** est une dégradation brutale de la qualité de service d'un flux, détectée par l'une des deux conditions suivantes : (a) le **score z robuste** de ln(délai) dépasse 3, calculé par type de tranche : z = (ln d − médiane de la tranche) / (1,4826 × MAD de la tranche) ; ou (b) un **quasi-effondrement** : plus de 90 % des paquets du flux sont perdus.

**Question du guide : Quelle mesure de changement, quel seuil, et pourquoi ?**

Nous mesurons le changement par l'écart au régime normal *du même type de tranche*. Cette précaution est indispensable : les tranches ont des niveaux de délai différents (figure 3), donc un seuil global confondrait la tranche URLLC rapide et la tranche eMBB lente. Le seuil retenu est de trois écarts-types robustes, ce qui équivaut à un test à environ trois sigmas ; nous utilisons la déviation absolue médiane (MAD) parce qu'elle est insensible aux valeurs extrêmes que nous cherchons précisément à détecter. Le seuil de 90 % de pertes cible quant à lui les flux qui n'écoulent presque plus de trafic.

Nous n'avons pas pu définir l'événement par une variation temporelle, puisqu'il n'existe pas de série temporelle dans ce jeu de données (les simulations sont indépendantes) : le « changement brusque » est donc un basculement de régime, du fonctionnement normal vers la saturation. Enfin, le masque d'événements ne sert qu'à découper l'évaluation ; il n'est jamais une entrée des modèles.

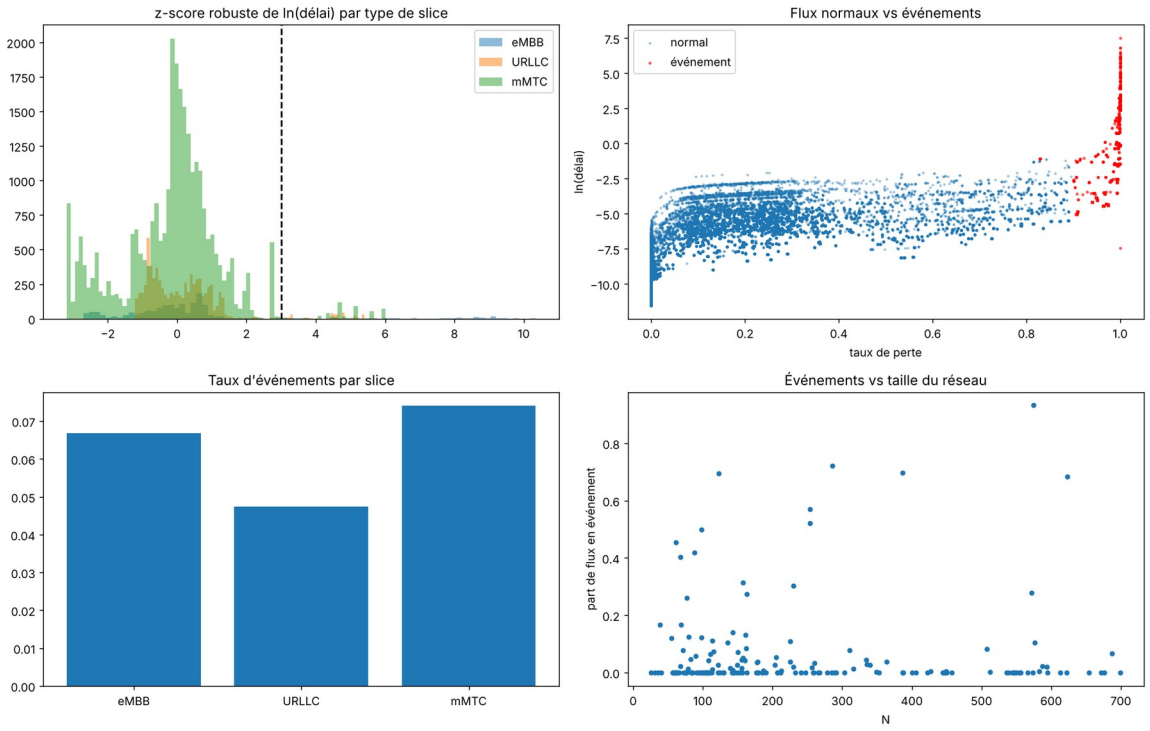



***Figure 6 –*** *Événements : score z par tranche (seuil en pointillé), flux normaux et événements dans le plan (perte, ln(délai)), taux par tranche et part de flux en événement selon N.*

## 7.2 Vérification visuelle

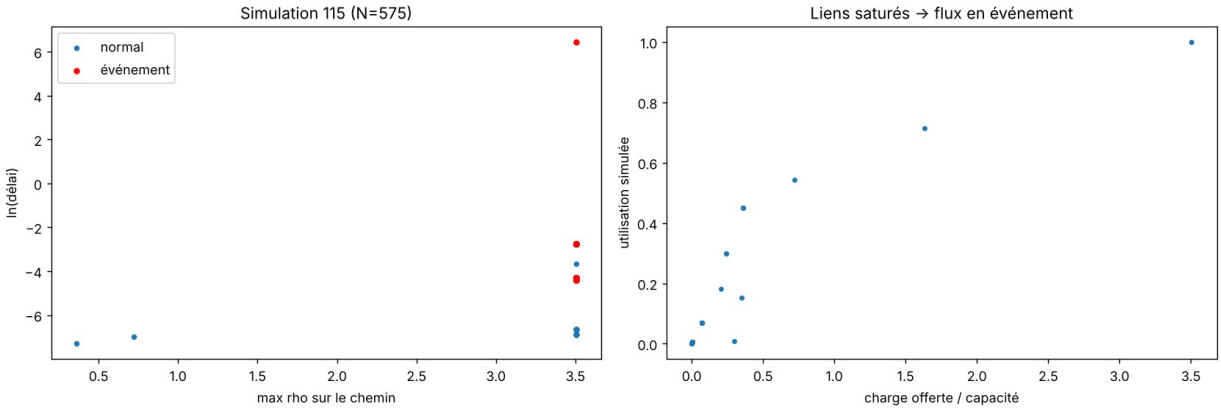



***Figure 7 –*** *Simulation 115 (N = 575), la plus touchée : les événements (en rouge) sont concentrés sur des flux dont le chemin traverse un lien en surcharge (ρ ≈ 3,5, utilisation simulée = 1).*

Les événements se séparent nettement dans le plan (perte, délai) : ils occupent la zone de perte proche de 1 et de délai élevé, là où les flux normaux ont un ln(délai) inférieur à −1. Dans la simulation 115, **tous les flux en événement passent par un lien saturé** (charge offerte égale à 3,5 fois la capacité, utilisation simulée de 100 %), alors que les liens de faible ρ ne portent que des flux normaux. Le masque correspond donc bien à un phénomène physique de congestion, ce qui valide notre définition.

## 7.3 Statistiques sur les événements

***Tableau 12 –*** *Événements, violations de SLA et pertes par type de tranche*

| Tranche | Nombre de flux | Taux d'événements | Violations de SLA (délai > δ) | Perte médiane |
|---|---|---|---|---|
| **URLLC** | 7 447 | 0,047 | 0,041 | 0,045 |
| **eMBB** | 1 527 | 0,067 | 0,077 | 0,242 |
| **mMTC** | 28 338 | 0,074 | 0,025 | 0,202 |

***Tableau 13 –*** *Taux d'événements selon le quartile de taille du réseau*

| Quartile de N | Nombre de flux | Taux d'événements |
|---|---|---|
| **(25,999 ; 225,0]** | 9 472 | 0,049 |
| **(225,0 ; 422,0]** | 9 294 | 0,091 |
| **(422,0 ; 572,0]** | 9 624 | 0,020 |
| **(572,0 ; 699,0]** | 8 922 | 0,118 |

Au total, **6,8 % des flux sont des événements** (environ 2 550 sur 37 312). Les flux mMTC (7,4 %) et eMBB (6,7 %) sont plus touchés que les flux URLLC (4,7 %) ; en revanche, la tranche eMBB est celle qui enregistre le plus de violations de SLA (7,7 %). Le taux d'événements n'est **pas monotone avec N** (de 2 % à 11,8 % selon le quartile) : il est dominé par quelques simulations très dégradées, dont certaines ont plus de 90 % de flux en événement. Par conséquent, les métriques « événements » reposent sur peu de simulations indépendantes et doivent être lues avec prudence.

# 8. L5 — Représentation en graphe et réseau de neurones sur graphes

## 8.1 Construction du graphe

Pour chaque simulation, nous construisons un graphe **hétérogène** qui contient trois types de nœuds : les flux, les files d'attente (un lien associé à une queue WFQ) et les liens. Les flux sont reliés aux files et aux liens qu'ils traversent d'après la matrice de routage, et chaque file est reliée au lien auquel elle appartient.

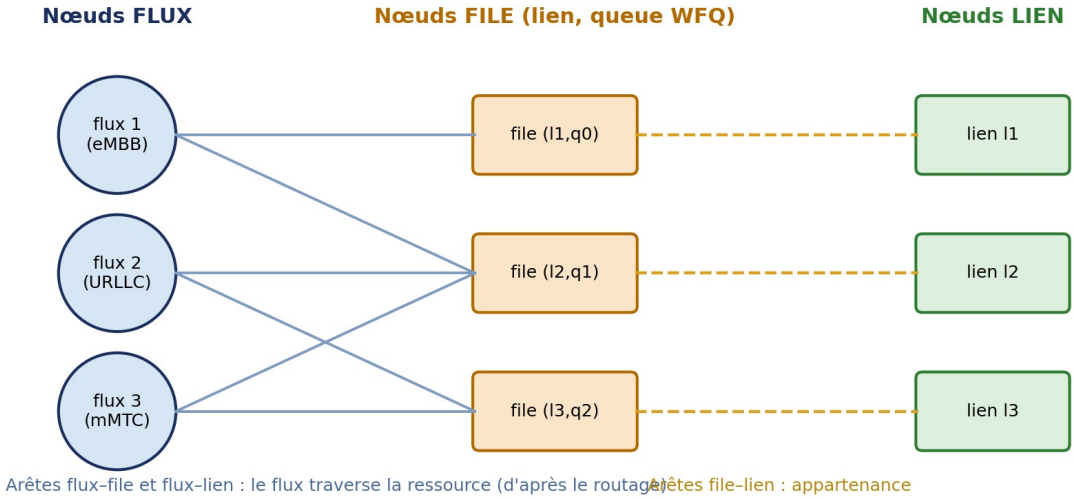



***Figure 8 –*** *Graphe hétérogène (un graphe par simulation) : nœuds flux, nœuds file (lien, queue WFQ) et nœuds lien.*

**Question du guide : Quels sont les nœuds, les arêtes et les caractéristiques ?**

Le graphe comporte trois types de nœuds. Les **nœuds flux** portent huit caractéristiques : le logarithme du débit, le logarithme de la taille de paquet, le logarithme du débit de paquets, le type de tranche codé en « one-hot » (trois valeurs), l'exigence de délai δ et le nombre de sauts. Les **nœuds lien** portent le logarithme de la capacité, le logarithme du degré et la politique WFQ ou FIFO. Les **nœuds file** portent le logarithme du degré, le logarithme de la part WFQ garantie et le logarithme de la capacité garantie.

Les arêtes sont non dirigées : elles relient un flux à chaque file qu'il traverse, un flux à chaque lien qu'il traverse (d'après la matrice de routage) et chaque file au lien auquel elle appartient.

**Question du guide : Le graphe est-il homogène ou hétérogène ?**

Le graphe est hétérogène, puisqu'il contient trois types de nœuds aux caractéristiques différentes. Nous l'avons implémenté comme un graphe homogène muni d'un encodeur de caractéristiques propre à chaque type de nœud : chaque type possède son bloc de caractéristiques dédié, masqué en dehors de son type. Cette implémentation permet d'utiliser les couches standard de PyTorch Geometric tout en respectant la nature hétérogène des données.

**Question du guide : Quelle est la sortie du modèle et quelle perte est utilisée ?**

Le modèle prédit ln(délai) pour chaque nœud flux. La perte est l'erreur quadratique moyenne calculée sur les nœuds flux uniquement ; les nœuds file et lien servent seulement de relais d'information. Par défaut, nous ne fournissons pas les caractéristiques de charge : le réseau doit apprendre la charge par agrégation. Nous avons testé en contrôle une variante « avec caractéristiques de charge ».

**Question du guide : Pourquoi un graphe est-il plus informatif qu'un simple vecteur de caractéristiques ?**

Le délai d'un flux dépend de la **contention** avec les autres flux qui partagent ses files : c'est une relation entre flux et non une propriété du flux seul. Dans le graphe, l'information du voisinage est agrégée automatiquement (flux,

puis file, puis autres flux), quelles que soient la taille et la topologie du réseau. Avec un vecteur, cette information ne peut être codée que par des agrégats choisis à la main (ρ maximal, ρ moyen, etc.), comme nous l'avons fait pour HistGradientBoosting. Le graphe offre aussi l'avantage d'accepter une taille variable, de 10 à environ 700 flux par simulation.

## 8.2 Architecture du réseau

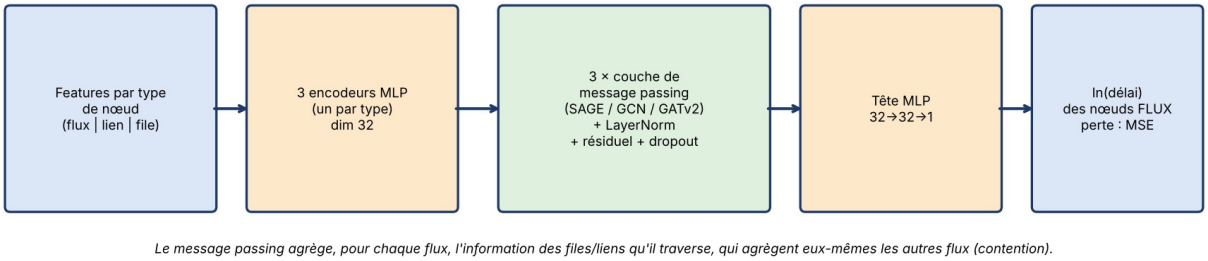



***Figure 9 –*** *Architecture du GNN : encodeurs par type de nœud, trois couches de passage de messages résiduelles et tête de régression.*

Le réseau fonctionne en quatre blocs. Trois encodeurs de type perceptron multicouche (un par type de nœud) projettent les caractéristiques dans un espace commun de dimension 32. Trois couches de **passage de messages** (message passing) agrègent ensuite, pour chaque flux, l'information des files et des liens qu'il traverse, qui agrègent eux-mêmes l'information des autres flux : c'est ainsi que la contention est apprise. Chaque couche est suivie d'une normalisation par couche (LayerNorm), d'une connexion résiduelle et d'un dropout. Enfin, une tête de régression (32 → 32 → 1) produit ln(délai) pour chaque nœud flux. En notation de code : x ← x + dropout(ReLU(LayerNorm(conv(x, arêtes)))).

## 8.3 Choix de l'architecture : GCN, GraphSAGE ou GATv2

**Question du guide : Quel type de GNN avons-nous choisi, et pourquoi ?**

Nous avons comparé trois couches de convolution sur graphe en relation avec la physique du problème, dans lequel la qualité de service dépend d'un **goulot d'étranglement** (le lien ou la file la plus chargée). La couche **GCN** calcule une moyenne normalisée par le degré : elle ne distingue pas les nœuds de degrés différents et lisse la charge, ce qui est inadapté à un goulot d'étranglement. La couche **GraphSAGE** utilise une agrégation moyenne *et* maximum : la moyenne mesure la charge typique des voisins, le maximum capte le voisin le plus chargé, et l'agrégation est invariante au nombre de voisins. La couche **GATv2** pondère les voisins par un mécanisme d'attention à quatre têtes, ce qui lui permet de donner plus de poids aux files congestionnées.

Les trois architectures ont été comparées avec une graine, et le choix final est fait sur la **MSE de validation**, jamais sur le test.

***Tableau 14 –*** *Comparaison des trois architectures de GNN (une graine)*

| Protocole | Architecture | MAE test | MSE test | MSE validation | Paramètres | Époques |
|---|---|---|---|---|---|---|
| **random** | SAGE | 1,153 | 3,076 | 3,210 | 14 305 | 85 |
| **random** | GCN | 1,496 | 4,613 | 6,378 | 8 161 | 39 |
| **random** | GATv2 | 1,217 | 2,987 | **2,885** | 11 521 | 79 |
| **shift** | SAGE | 1,477 | 3,761 | 1,797 | 14 305 | 72 |
| **shift** | GCN | 2,436 | 9,249 | 3,763 | 8 161 | 111 |
| **shift** | GATv2 | 2,087 | 6,511 | **1,670** | 11 521 | 102 |

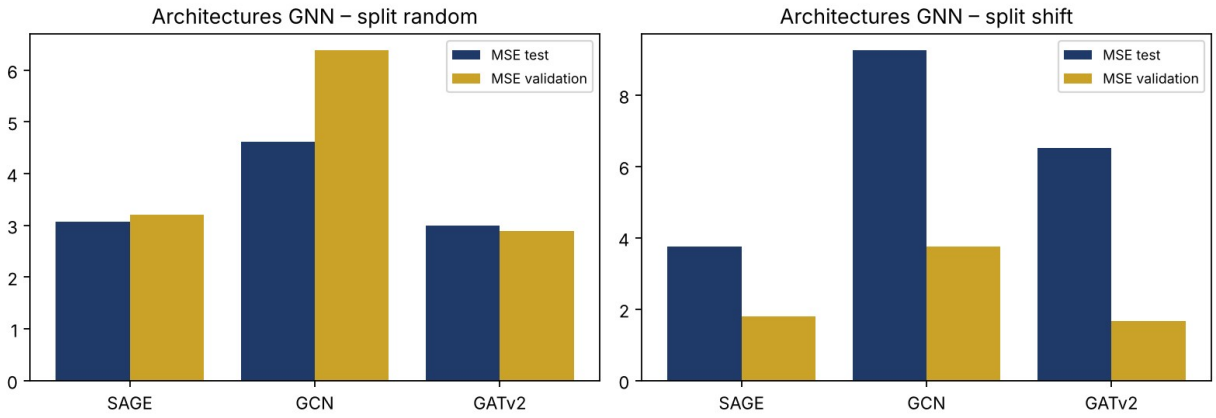



***Figure 10 –*** *Comparaison des architectures de GNN : MSE de test et de validation pour les protocoles « random » et « shift ».*

GATv2 obtient la meilleure MSE de validation dans les deux protocoles (2,885 en « random », 1,670 en « shift ») et nous la retenons, avec trois graines. La couche GCN est nettement la moins bonne (MSE de test de 4,61 en « random » et 9,25 en « shift »), ce qui confirme que l'agrégation par moyenne normalisée perd l'information de goulot.

> **Remarque honnête.** En protocole « shift », GraphSAGE (moyenne et maximum) est en réalité meilleur sur le test (MSE de 3,76) que GATv2 (6,51), alors que la validation, composée de petits réseaux, ne pouvait pas le révéler. La sélection sur la validation est correcte sur le plan méthodologique, mais elle illustre qu'une validation de même distribution que l'entraînement est **aveugle au changement de distribution**. Nous revenons sur ce point dans les limites (section 10).

## 8.4 Hyperparamètres

***Tableau 15 –*** *Hyperparamètres du GNN retenu et leur justification*

| Hyperparamètre | Valeur | Justification |
|---|---|---|
| **Dimension cachée / couches** | 32 / 3 | Trois sauts correspondent au rayon de contention (flux → file → flux voisin → …) ; petit modèle de 8 000 à 14 000 paramètres, adapté aux 108 graphes d'entraînement. |
| **Dropout, décroissance des poids** | 0,2 ; $10^{-3}$ (optimiseur AdamW) | Régularisation contre le surapprentissage. |
| **Taux d'apprentissage** | $3{\cdot}10^{-3}$, réduit de moitié après 8 époques sans amélioration (ReduceLROnPlateau) | Stabilité de la courbe de validation. |
| **Époques / patience** | 120 maximum / 25 (arrêt anticipé sur la MSE de validation) | Conserve le meilleur état de validation. |
| **Taille de lot** | 8 simulations (graphes entiers) | Un lot est un mini-ensemble de graphes complets. |
| **Autres** | LayerNorm, connexions résiduelles, écrêtage du gradient (5), standardisation par type de nœud calculée sur l'entraînement, cible standardisée | Stabilité de l'entraînement et lutte contre le sur-lissage des GNN profonds. |

## 8.5 Contrôle du surapprentissage

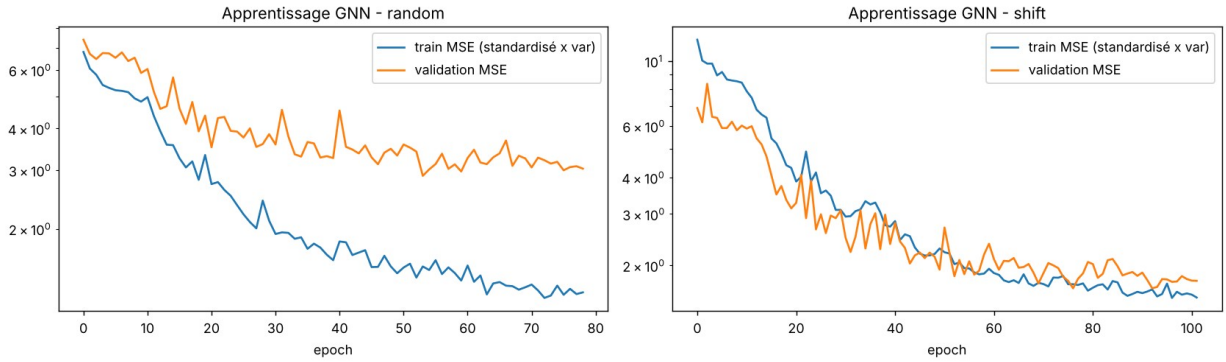



***Figure 11 –*** *Courbes d'apprentissage du GNN retenu : MSE d'entraînement et de validation (échelle logarithmique).*

**Question du guide : Comment avons-nous contrôlé le surapprentissage ?**

Nous avons contrôlé le surapprentissage par quatre moyens : le dropout (0,2), la décroissance des poids ($10^{-3}$), l'arrêt anticipé avec conservation de l'état de meilleure validation, et un modèle volontairement petit. En protocole « random », l'écart entre l'entraînement (MSE d'environ 1,1) et la validation (environ 3,0) s'ouvre : il s'agit d'un surapprentissage modéré, normal avec seulement 108 graphes, et il est borné par ces mécanismes.

En protocole « shift », les courbes d'entraînement et de validation restent proches (environ 1,6 à 1,7) : il n'y a pas de surapprentissage intra-distribution. L'échec du GNN sous changement de distribution (section 9) provient donc du **décalage de distribution** et non d'un manque de régularisation.

## 8.6 Justification des choix de conception (code, graphe, modèles)

Cette section justifie les choix de code, la construction du graphe (nœuds, arêtes, caractéristiques) et le choix des modèles, en particulier GraphSAGE, GATv2 et GCN. Chaque choix répond à un risque précis : fuite d'information, manque de données ou confusion entre deux effets. Pour chacun, nous indiquons ce qu'il coûte.

### 8.6.1 Pourquoi ces choix de code

***Tableau 15 bis –*** *Choix de code, justification et limite*

| Choix | Pourquoi | Limite ou coût |
|---|---|---|
| **Chemin routé (matrice de routage), pas le chemin planifié des tranches** | La corrélation avec l'utilisation réelle des liens est de 0,93 à 0,98 pour le routage, contre 0,4 à 0,7 pour le plan : les charges ρ sont calculées sur le chemin réellement simulé | Les caractéristiques « planifiées » restent dans un groupe à part ; leur retrait améliore le résultat en shift |
| **Aucune sortie du simulateur en entrée (délai, pertes, utilisation simulée)** | Ce sont des résultats de simulation : les fournir en entrée donnerait la réponse au modèle (fuite d'information) | Le modèle ne voit que la congestion attendue (débits offerts / capacités), pas la congestion observée |
| **Découpage par simulation, jamais par flux** | Les flux d'une simulation partagent les mêmes liens et files : un découpage par flux ferait fuir l'information de contention vers le test | Seulement 165 simulations, donc 108 graphes indépendants pour l'entraînement |
| **Cible ln(délai)** | Le délai varie de 4 µs à 1825 s ; le logarithme donne une erreur relative et stabilise la perte quadratique | Les erreurs en échelle logarithmique ne se lisent pas directement en secondes |
| **Standardisation des caractéristiques et de la cible calculée sur le train uniquement** | Le jeu de test ne doit influencer ni les moyennes ni les écarts-types | Aucune |

| Choix | Pourquoi | Limite ou coût |
|---|---|---|
| **Python, scikit-learn, PyTorch, PyTorch Geometric** | scikit-learn fournit Ridge et HistGradientBoosting ; PyTorch Geometric fournit les trois couches comparées (SAGEConv, GCNConv, GATv2Conv) et un DataLoader qui regroupe des graphes de tailles différentes (de 10 à environ 700 flux) | Nous n'avons pas comparé d'autres bibliothèques de GNN |
| **Graphe hétérogène codé comme un graphe homogène : type de nœud `ntype`, un encodeur par type, colonnes masquées en dehors du type** | Permet d'utiliser les couches standard tout en gardant des caractéristiques différentes pour les flux, les liens et les files | Les couches partagent les mêmes poids pour toutes les relations (flux–file, flux–lien, file–lien) : le modèle ne distingue les relations que par les caractéristiques des nœuds |
| **Perte calculée uniquement sur les nœuds flux (masque `is_flow`)** | Seuls les flux ont une cible ; liens et files servent de relais d'information | Aucune |
| **Modèle petit (dimension 32, 3 couches, 8 000 à 14 000 paramètres), exécution sur CPU** | Avec 108 graphes d'entraînement, un grand modèle surapprendrait ; un GPU n'est pas nécessaire | Capacité d'expression limitée |
| **Mise en cache (tables CSV, `graphs.pkl`, `flows_features.pkl`)** | L'extraction des 33 archives n'est faite qu'une fois ; les scripts suivants relisent les tables | Supprimer `flows_features.pkl` après toute modification des caractéristiques |
| **Graine 42, puis 3 graines pour le GNN retenu** | Reproductibilité et mesure de la variabilité entre entraînements | La comparaison des architectures et les ablations n'utilisent qu'une graine |

### 8.6.2 Pourquoi ces nœuds, ces arêtes et ces caractéristiques

Le graphe est construit pour représenter la contention : le délai d'un flux dépend des autres flux qui partagent ses ressources. Il faut donc des nœuds pour les flux et pour les ressources qu'ils se disputent.

***Tableau 15 ter –*** *Les trois types de nœuds*

| Nœud | Ce qu'il représente | Caractéristiques | Pourquoi ce choix |
|---|---|---|---|
| **Flux** | Une communication source → destination appartenant à une tranche ; c'est l'objet dont on prédit ln(délai) | ln du débit, ln de la taille de paquet, ln du débit de paquets, type de tranche (one-hot, 3 valeurs), exigence de délai δ, nombre de sauts (8 caractéristiques) | Ce sont les entrées de demande et de service propres au flux ; le logarithme compresse des échelles très différentes (de 0,2 à 18 Mbit/s) |
| **Lien** | Une liaison dirigée du réseau | ln de la capacité, ln du degré, politique d'ordonnancement (WFQ ou FIFO) | La capacité borne le débit écoulé ; le degré (nombre de voisins dans le graphe) mesure combien de flux et de files passent par le lien ; WFQ ou FIFO change le partage |
| **File** | Une file d'attente WFQ d'un lien, identifiée par le couple (lien, queue) | ln du degré, ln de la part WFQ garantie, ln de la capacité garantie | En WFQ, un flux obtient la part garantie de sa file, pas toute la capacité du lien : deux flux sur un même lien mais dans des files différentes ne se concurrencent pas de la même façon |

**Les arêtes.** Trois relations sont créées : flux–file (le flux traverse la file), flux–lien (le flux traverse le lien, d'après le routage simulé) et file–lien (la file appartient au lien). Elles sont non dirigées : l'information circule du flux vers la ressource (sa charge), puis de la ressource vers les autres flux (la congestion qu'ils subissent). Deux sauts suffisent pour qu'un flux « voie » ses concurrents directs (flux → file → autre flux) ; le troisième transmet l'état de ces concurrents sur leurs autres ressources.

**Pourquoi pas une autre représentation.**

- **Un vecteur par flux.** La contention ne peut y être codée que par des agrégats choisis à la main (ρ maximal, ρ moyen, part garantie…), ce qui suppose de connaître le routage et la formule de la charge. C'est exactement ce que fait HistGradientBoosting.
- **Un graphe de la topologie physique seule (routeurs et liens).** Le délai est une grandeur par flux ; les flux devraient alors être résumés dans les caractéristiques des nœuds, ce qui perd la notion de concurrents précis.
- **Un graphe flux–lien sans nœuds file.** Il ignorerait le partage WFQ. Nous avons gardé les files parce que, pour HistGradientBoosting, le groupe « files / WFQ » est décisif : sa suppression fait passer la MSE de 2,12 à 4,85 en protocole random.

> **Nuance à connaître.** Les nœuds file existent, mais le GNN exploite peu les poids WFQ : retirer la configuration WFQ ne dégrade pas significativement son résultat (3,04 contre 3,15 ± 0,24). Le graphe contient donc l'information, sans que le modèle l'utilise pleinement avec 108 graphes. Les arêtes n'ont pas de caractéristiques et l'ordre des sauts le long du chemin n'est pas codé : ce sont deux limites de notre représentation.

### 8.6.3 Pourquoi ces modèles

Chaque modèle répond à une question précise ; aucun n'est choisi parce qu'il est populaire.

***Tableau 15 quater** – Rôle de chaque modèle et résultat (MSE sur ln(délai), jeu de test)*

| Modèle | Principe | Question à laquelle il répond | MSE random / shift |
|---|---|---|---|
| **mean_train** | Prédit la moyenne de ln(délai) du train | Qu'obtient-on sans aucune information ? | 5,29 / 4,79 |
| **Ridge** | Régression linéaire régularisée, sans information de congestion | Un modèle linéaire simple suffit-il ? (H1) | 4,93 / 4,30 |
| **HistGradientBoosting** | Arbres boostés avec la charge ρ calculée à la main | Un modèle non linéaire bien nourri capte-t-il la rupture de régime autour de ρ ≈ 1 ? (H2) | 2,12 / 3,37 |
| **MLP (GNN sans arêtes)** | Mêmes caractéristiques individuelles, aucun échange entre nœuds | Le graphe apporte-t-il quelque chose en lui-même ? (H3) | 4,99 / 5,60 |
| **GCN** | Moyenne des voisins normalisée par les degrés | Une agrégation qui lisse la charge convient-elle ? | 4,61 / 9,25 |
| **GraphSAGE (moyenne + maximum)** | Concatène la moyenne et le maximum des messages des voisins | Conserver le voisin le plus chargé aide-t-il ? | 3,08 / 3,76 |
| **GATv2 (4 têtes d'attention)** | Pondère chaque voisin par un poids appris | Donner plus de poids aux files congestionnées aide-t-il ? | 2,99 / 6,51 (3 graines : 3,15 / 6,38) |

*Les valeurs GCN, GraphSAGE et GATv2 sont celles du tableau 14 (une graine) ; la ligne MLP vient du tableau 18.*

#### Pourquoi l'agrégation compte pour un goulot d'étranglement

Le délai d'un flux est gouverné par son lien ou sa file la plus chargée. Prenons, à titre d'illustration, un flux dont le chemin traverse quatre ressources de charge ρ = 3,5 ; 0,2 ; 0,2 et 0,2. Le **maximum** vaut 3,5 et signale la saturation. La **moyenne** vaut 1,025 : elle ressemble à une charge « limite » et masque que l'une des ressources est saturée à 350 %. Le choix de l'agrégation décide donc de l'information qui parvient au flux.

- **GCN** calcule une moyenne pondérée par les degrés des voisins : la charge d'un voisin très chargé est diluée parmi les autres. C'est le comportement le moins adapté à un goulot.

- **GraphSAGE** reçoit la **moyenne** (charge typique) et le **maximum** (voisin le plus chargé), concaténés puis projetés. Le maximum conserve directement le goulot, quel que soit le nombre de voisins.
- **GATv2** calcule un score d'attention par voisin puis une somme pondérée, avec 4 têtes. Il peut apprendre à donner plus de poids aux files saturées. GATv2 est la version corrigée de GAT : son classement des voisins dépend du nœud qui les reçoit, alors que dans GAT d'origine il est le même pour tous.

#### Pourquoi GATv2 a été retenu, et pourquoi rester prudent

La règle de sélection a été fixée avant les expériences : l'architecture est choisie sur la **MSE de validation**, jamais sur le test. GATv2 a la meilleure validation dans les deux protocoles (2,885 en random et 1,670 en shift) ; il est donc retenu et entraîné avec 3 graines. Trois précisions évitent de surinterpréter ce choix :

- **En random, GraphSAGE et GATv2 sont équivalents.** Sur le test, 3,076 contre 2,987 : l'écart est inférieur à la variabilité entre graines du GNN (± 0,24).
- **En shift, GraphSAGE est meilleur sur le test** (3,76 contre 6,51), mais la validation, composée de petits réseaux, ne pouvait pas le révéler.
- **L'interprétation de l'échec de GCN par le lissage est plausible, mais elle n'est pas isolée.** GCN a aussi moins de paramètres (8 161 contre 14 305 et 11 521) et s'arrête plus tôt en random (39 époques contre 85 et 79). Nous n'avons pas séparé l'effet de l'agrégation seule de celui de la taille du modèle.

#### Pourquoi HistGradientBoosting reste meilleur que le GNN

HistGradientBoosting reçoit des caractéristiques de charge construites avec notre connaissance du routage et de WFQ : elles décrivent directement la congestion et sont bornées, donc plus faciles à transférer vers de grands réseaux. Le GNN doit découvrir cette information par agrégation, avec seulement 108 graphes. Quand on lui fournit ces mêmes caractéristiques, il atteint le niveau de HistGradientBoosting en random (2,116 contre 2,119) : l'écart ne vient donc pas d'une incapacité du graphe, mais de l'information qu'il doit reconstruire.

#### Modèles non testés

Le guide demande de choisir parmi GCN, GraphSAGE, GAT ou une autre architecture pertinente. Nous n'avons pas testé :

- **GAT d'origine**, remplacé par GATv2 pour la raison ci-dessus ;
- **GIN**, dont l'agrégation par **somme** conserve le nombre de voisins et la charge cumulée, ce qui correspond bien à une charge qui s'additionne : c'est une piste naturelle ;
- **des modèles qui font circuler l'information le long du chemin** (par exemple avec un réseau récurrent sur la suite des sauts), qui coderaient l'ordre des sauts que notre graphe ignore.

### 8.6.4 Synthèse des choix défendus

- **Aucune fuite d'information :** routage simulé, aucune sortie du simulateur en entrée, découpage par simulation, standardisation sur le train.
- **Un graphe fidèle à la physique :** flux, liens et files, parce que la contention se joue dans les files WFQ partagées.
- **Des modèles choisis pour une question précise :** moyenne et Ridge comme références, HistGradientBoosting comme référence forte, MLP comme contrôle de l'apport du graphe, GCN / GraphSAGE / GATv2 pour comparer des façons d'agréger la charge.
- **Un choix d'architecture honnête :** GATv2 est retenu sur la validation, mais GraphSAGE fait aussi bien en random et mieux en shift ; la comparaison n'utilise qu'une graine et ne sépare pas l'effet de l'agrégation de celui de la taille du modèle.

# 9. L6 — Comparaison, ablations, robustesse et analyse

## 9.1 Comparaison principale sur le jeu de test

*Tableau 16 – Comparaison principale des quatre modèles (test) ; en gras, le meilleur modèle de chaque protocole*

| Protocole | Modèle | MAE globale | MSE globale | MAE événements | MSE événements |
|---|---|---|---|---|---|
| random | mean_train | 1,730 | 5,292 | 4,027 | 22,687 |
| random | ridge_base | 1,643 | 4,930 | 4,310 | 24,680 |
| random | hgb_all | **0,967** | **2,119** | **1,333** | **5,658** |
| random | gnn_best | 1,287 | 3,154 | 2,477 | 10,547 |
| shift | mean_train | 1,541 | 4,789 | 4,053 | 22,325 |
| shift | ridge_base | 1,466 | 4,302 | 4,327 | 24,243 |
| shift | hgb_all | **1,390** | **3,374** | **1,264** | **2,798** |
| shift | gnn_best | 2,065 | 6,379 | 3,714 | 17,600 |

Pour le GNN, nous donnons la moyenne de trois graines : en protocole « random », MAE de 1,287 ± 0,081 et MSE de 3,154 ± 0,244 ; en protocole « shift », MAE de 2,065 ± 0,094 et MSE de 6,379 ± 0,714 (sur les événements : 10,55 ± 2,24 et 17,60 ± 4,37).

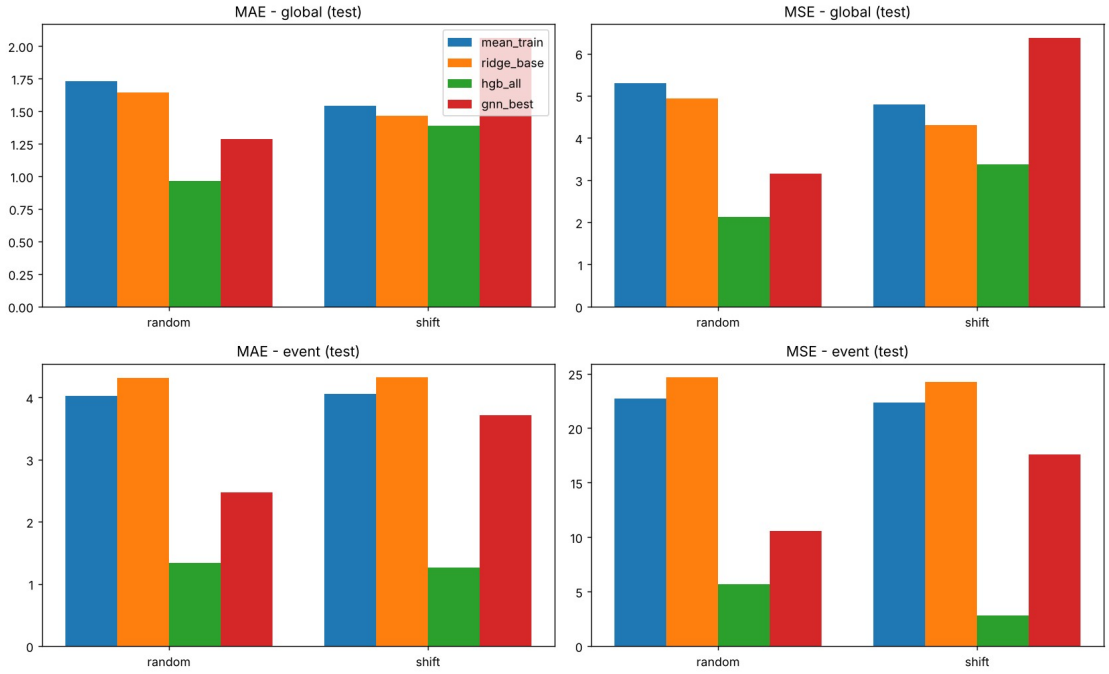



*Figure 12 – MAE et MSE : évaluation globale (haut) et sur les événements (bas), protocoles « random » et « shift ».*

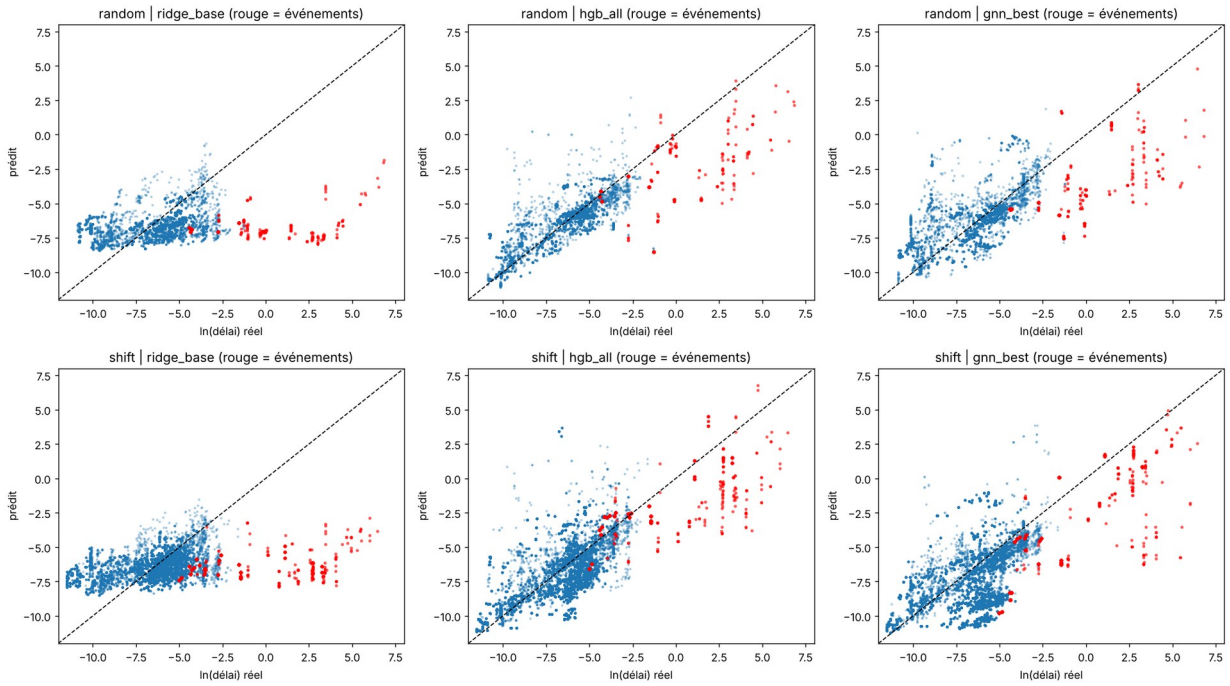



***Figure 13 –*** *Valeurs prédites en fonction des valeurs réelles pour chaque modèle et chaque protocole (rouge : événements ; pointillé : prédiction parfaite).*

### Lecture des résultats

- **Protocole « random ».** HistGradientBoosting est le meilleur (MSE de 2,12 ; 5,66 sur les événements). Le GNN bat nettement la référence Ridge (−36 % de MSE globale, −57 % sur les événements) mais reste moins bon que HistGradientBoosting (3,15 contre 2,12). Le GNN a donc bien appris la contention *sans* recevoir les caractéristiques de charge.
- **Prédit contre réel.** Ridge prédit une valeur quasi constante (autour de −7) : il rate toute la dynamique. HistGradientBoosting et le GNN suivent la diagonale pour les flux normaux. Pour les événements, **tous les modèles sous-estiment** les délais extrêmes (points rouges sous la diagonale) ; le GNN présente une dispersion plus forte.
- **Protocole « shift ».** Le GNN se dégrade fortement : sa MSE de 6,38 est pire que celle de la moyenne du train (4,79) et que celle de Ridge (4,30). HistGradientBoosting reste le plus fiable (3,37).

## 9.2 Robustesse au changement de distribution

***Tableau 17 –*** *Robustesse au changement de distribution : MSE en « random » et en « shift »*

| Modèle | Sous-ensemble | MSE random | MSE shift | Rapport shift / random |
|---|---|---|---|---|
| **mean_train** | global | 5,29 | 4,79 | 0,91 |
| **mean_train** | événements | 22,69 | 22,32 | 0,98 |
| **ridge_base** | global | 4,93 | 4,30 | 0,87 |
| **ridge_base** | événements | 24,68 | 24,24 | 0,98 |
| **hgb_all** | global | 2,12 | 3,37 | 1,59 |
| **hgb_all** | événements | 5,66 | 2,80 | 0,49 |
| **gnn_best** | global | 3,15 | 6,38 | 2,02 |
| **gnn_best** | événements | 10,55 | 17,60 | 1,67 |

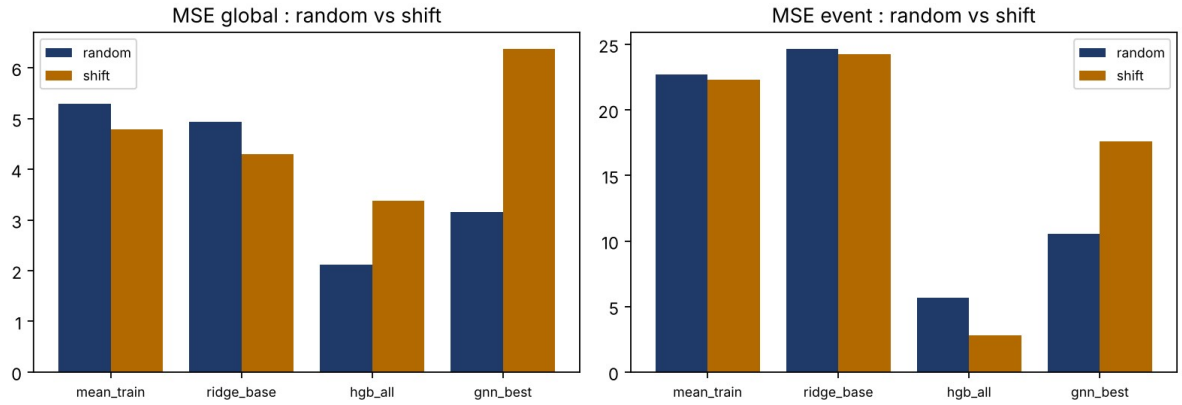



***Figure 14 –*** *MSE globale et MSE sur les événements : protocole « random » contre protocole « shift ».*

**Analyse.** Les modèles triviaux (moyenne, Ridge) varient peu parce qu'ils ne prédisent presque rien d'informatif. HistGradientBoosting se dégrade d'un facteur 1,6 en global ; le GNN d'un facteur 2,0 en global et 1,7 sur les événements. Le rapport de 0,495 de HistGradientBoosting sur les événements **ne doit pas être sur-interprété** : les ensembles d'événements des deux protocoles sont différents (757 flux contre 1 775, issus de simulations distinctes et très inégalement touchées), et HistGradientBoosting y obtient simplement une MSE faible (2,80) sur le test « shift ».

Nous proposons deux causes plausibles pour la dégradation du GNN. (i) Il dispose de moins de données d'entraînement : 11 693 flux seulement, issus de petits réseaux. (ii) Les grands réseaux ont des chemins, des degrés de file et des contentions hors du domaine vu à l'entraînement : le GNN doit *extrapoler son agrégation*, alors que HistGradientBoosting s'appuie sur des caractéristiques physiques bornées (ρ, part garantie) qui se transfèrent mieux d'une taille de réseau à l'autre. Ces deux causes ne sont pas séparées par notre protocole : la taille du réseau et la quantité de données d'entraînement varient en même temps.

## 9.3 Ablations

Une ablation consiste à retirer un groupe d'information pour mesurer sa contribution. Nous l'avons appliquée aux deux familles de modèles : par groupe de caractéristiques pour HistGradientBoosting, et par composant du graphe pour le GNN.

***Tableau 18 –*** *Résultats des ablations (test)*

| Modèle | MAE random | MAE shift | MSE random | MSE shift |
|---|---|---|---|---|
| **HGB – toutes caractéristiques** | 0,967 | 1,390 | 2,119 | 3,374 |
| **HGB – trafic** | 0,967 | 1,390 | 2,119 | 3,374 |
| **HGB – tranche** | 0,981 | 1,426 | 2,040 | 3,555 |
| **HGB – structure du chemin** | 0,933 | 1,427 | 1,982 | 3,459 |
| **HGB – charge du chemin** | 1,756 | 2,696 | 5,585 | 11,979 |
| **HGB – files / WFQ** | 1,452 | 1,907 | 4,852 | 6,373 |
| **HGB – planifié** | 0,942 | 1,324 | 2,113 | 3,138 |
| **HGB – contexte** | 0,912 | 1,433 | 1,990 | 3,780 |
| **GNN (GATv2) complet** | 1,287 | 2,065 | 3,154 | 6,379 |
| **GNN sans arêtes (MLP)** | 1,548 | 1,746 | 4,990 | 5,601 |
| **GNN – information de tranche** | 1,392 | 2,042 | 3,521 | 6,212 |
| **GNN – configuration WFQ** | 1,239 | 1,741 | 3,043 | 5,370 |
| **GNN + caractéristiques de charge** | 1,025 | 1,847 | 2,116 | 5,037 |

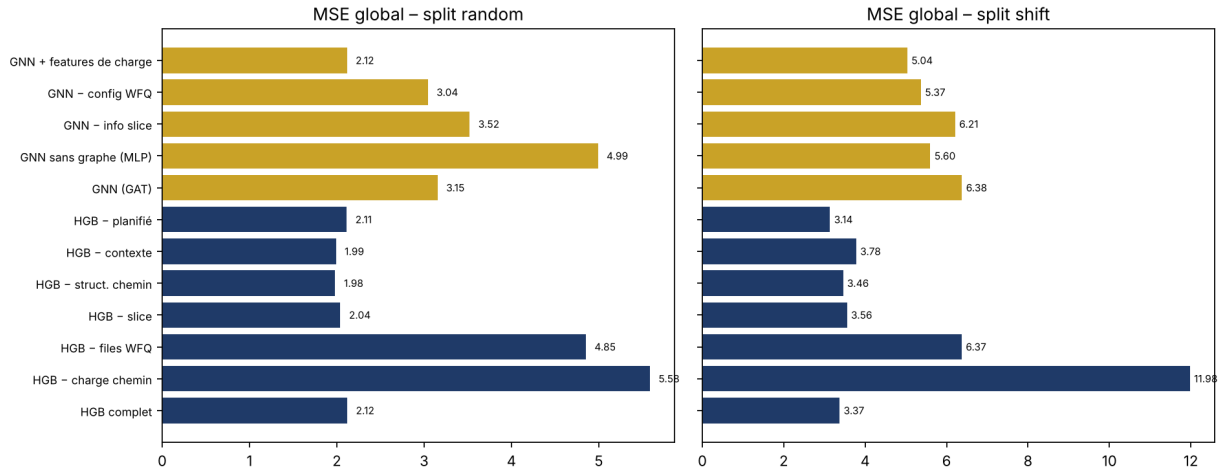



***Figure 15 –*** *Ablations : MSE globale par modèle (bleu : HGB ; jaune : GNN) pour les deux protocoles.*

## Ablation des caractéristiques (HistGradientBoosting)

- **La charge du chemin et la configuration des files / WFQ sont décisives.** Leur retrait fait passer la MSE de 2,12 à 5,59 et 4,85 (« random ») et de 3,37 à 11,98 et 6,37 (« shift »). C'est la confirmation quantitative de H1 : la congestion et la configuration d'ordonnancement déterminent la qualité de service.
- **Tranche, structure du chemin et contexte** changent peu le résultat (de l'ordre de ±0,1) et l'améliorent même légèrement en « random » : ces informations sont redondantes avec la charge. Le retrait des caractéristiques **planifiées** améliore le résultat en « shift » (3,14 contre 3,37), ce qui s'explique : elles correspondent à un plan différent du routage simulé et sont donc potentiellement trompeuses.
- **Point d'attention sur le groupe « trafic ».** Son retrait donne des résultats strictement identiques à ceux du modèle complet (MSE de 2,119 et 3,374). Les trois caractéristiques de trafic étant très redondantes entre elles et avec les charges (qui intègrent déjà les débits), une identité parfaite suggère qu'elles ne sont jamais sélectionnées par les arbres retenus ; or l'importance par permutation attribue un poids à `log_pkts_lambda` (section 9.4). Cette contradiction apparente indique que l'ablation de ce groupe ne doit pas être interprétée comme « le trafic est inutile » : la cause la plus probable est une redondance totale avec les caractéristiques de charge ou une correspondance incomplète entre le nom du groupe et les colonnes retirées. Nous recommandons de vérifier la liste des colonnes retirées et de relancer cette ablation avant la soutenance ; cette vérification ne change pas nos conclusions.

## Ablation du graphe (GNN)

- **H3 confirmée en « random ».** Retirer le passage de messages (un perceptron multicouche sur les mêmes caractéristiques individuelles) fait passer la MSE de 3,15 à 4,99 (+58 %), et de 10,5 à 25,9 sur les événements, où le MLP n'est pas meilleur que Ridge. Le graphe apporte donc bien l'information de contention.
- **Mais sous « shift », l'avantage disparaît** : 6,38 pour le GNN contre 5,60 pour le MLP. Le graphe n'apporte plus de gain là où il est théoriquement le plus utile.
- **Ajout des caractéristiques de charge calculées à la main.** La MSE du GNN tombe à 2,116 en « random », identique à celle de HistGradientBoosting (2,119) : quand la congestion est fournie explicitement, le GNN n'a plus d'avantage structurel, alors que sans elle il en retrouve environ 60 % par la seule agrégation. En « shift » (5,04), il reste moins bon que HistGradientBoosting (3,37).
- **Information de tranche et configuration WFQ.** Retirer l'information de tranche dégrade le résultat (3,52 ; 6,21 en « shift »). Retirer la configuration WFQ ne le dégrade pas de façon significative en « random » (3,04 contre 3,15 ± 0,24, dans la variabilité entre graines) : le GNN peine à exploiter les seuls poids WFQ, alors que HistGradientBoosting les utilise fortement, via le rapport débit garanti / demande et ρ par file.

## 9.4 Importance des caractéristiques et erreurs par groupe

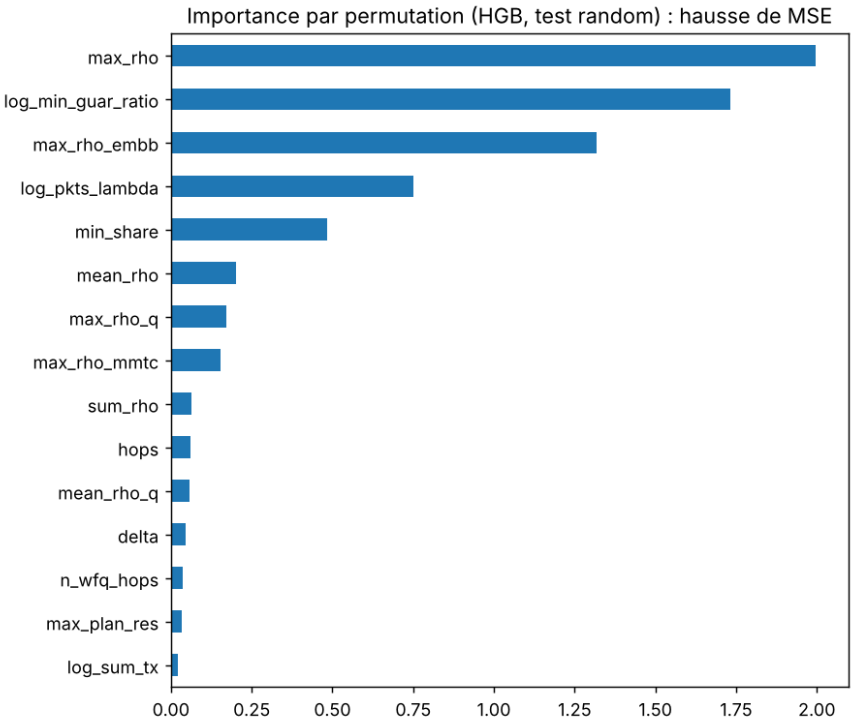



***Figure 16 –*** *Importance par permutation (HistGradientBoosting, test « random ») : hausse de la MSE quand la caractéristique est mélangée.*

***Tableau 19 –*** *Caractéristiques les plus importantes par permutation*

| Caractéristique | Hausse de la MSE (permutation) |
|---|---|
| **max_rho** | 2,00 |
| **log_min_guar_ratio** | 1,73 |
| **max_rho_embb** | 1,32 |
| **log_pkts_lambda** | 0,75 |
| **min_share** | 0,48 |
| **mean_rho** | 0,20 |
| **max_rho_q** | 0,17 |
| **max_rho_mmtc** | 0,15 |

Les caractéristiques les plus importantes sont la charge maximale sur le chemin (hausse de 2,0 de MSE si on la permute), le rapport débit garanti / débit demandé (1,73), la charge maximale de la tranche eMBB (1,32), le débit de paquets (0,75) et la part WFQ minimale (0,48). **La qualité de service est donc gouvernée par le goulot d'étranglement du chemin, par la charge de la classe eMBB (la plus gourmande) et par la part garantie par WFQ.**

***Tableau 20 –*** *Erreurs et coefficients de détermination par groupe (protocole « shift »)*

| Groupe (test « shift ») | MAE ridge | MAE hgb | MAE gnn | $R^2$ ridge | $R^2$ hgb | $R^2$ gnn |
|---|---|---|---|---|---|---|
| **tranche eMBB** | 1,340 | 0,973 | 1,228 | −0,001 | 0,525 | 0,324 |
| **tranche URLLC** | 1,597 | 0,995 | 1,080 | 0,130 | 0,452 | 0,559 |
| **tranche mMTC** | 1,452 | 1,455 | 2,248 | 0,050 | 0,218 | −0,639 |
| **N petit** | 1,734 | 1,370 | 1,668 | 0,195 | 0,464 | 0,278 |

| Groupe (test « shift ») | MAE ridge | MAE hgb | MAE gnn | $R^2$ ridge | $R^2$ hgb | $R^2$ gnn |
|---|---|---|---|---|---|---|
| **N moyen** | 1,281 | 1,623 | 2,422 | −0,019 | −0,391 | −1,566 |
| **N grand** | 1,374 | 1,175 | 2,185 | −0,148 | 0,386 | −0,826 |

**Erreurs par groupe.** Le GNN est meilleur que HistGradientBoosting sur la tranche URLLC en MSE (2,79 contre 3,47 ; $R^2$ de 0,56 contre 0,45), mais il est très mauvais sur la tranche mMTC (MAE de 2,25 contre 1,46 ; $R^2$ de −0,64) qui représente 86 % des flux de test, et sur les réseaux de taille moyenne et grande ($R^2$ négatif). Les flux mMTC, très nombreux et de faible débit, situés dans de grands réseaux, sont ceux que le GNN extrapole le moins bien.

Cette analyse complète H4 : les performances se dégradent bien sous changement de distribution pour les modèles qui apprennent (HGB et GNN), et les événements sont plus difficiles à prédire que les cas normaux (la MSE sur les événements est supérieure à la MSE globale) pour HistGradientBoosting et pour le GNN en « random », ainsi que pour le GNN en « shift ». Seul HistGradientBoosting en « shift » fait exception, pour la raison de composition des ensembles d'événements expliquée plus haut ; nous considérons donc H4 comme **confirmée avec réserve**.

## 9.5 Tableau de synthèse comparative

*Tableau 21 – Synthèse comparative des trois approches*

| Critère | Ridge (référence) | HistGradientBoosting | GNN (GATv2) |
|---|---|---|---|
| **MSE globale, « random »** | 4,93 | **2,12** | 3,15 |
| **MSE globale, « shift »** | 4,30 | **3,37** | 6,38 |
| **MSE événements, « random » / « shift »** | 24,7 / 24,2 | **5,66 / 2,80** | 10,5 / 17,6 |
| **Besoin de caractéristiques de charge conçues à la main** | Non (mais inefficace) | **Oui** (indispensables) | Non (apprises par agrégation) |
| **Sensibilité à la taille du réseau** | Faible (ne prédit presque rien) | Modérée (×1,6) | Forte (×2,0) |
| **Qualités principales** | Simplicité | Précision et robustesse | Généricité de la représentation |

## 9.6 Réponses aux questions de l'étape 8

### Question du guide : Quel modèle généralise le mieux ?

Le modèle HistGradientBoosting généralise le mieux : son erreur quadratique moyenne est de 2,12 avec le protocole « random » et de 3,37 avec le protocole « shift ». C'est le seul modèle qui reste meilleur que les modèles de référence dans les deux protocoles, alors que le réseau de neurones sur graphes se retrouve loin derrière lorsque la taille des réseaux change.

### Question du guide : Quel modèle résiste le mieux aux événements ?

HistGradientBoosting résiste le mieux aux événements, avec une erreur quadratique moyenne de 5,66 en protocole « random » et de 2,80 en protocole « shift ». Le réseau de neurones sur graphes obtient 10,5 puis 17,6, et la régression Ridge 24,7 puis 24,2, c'est-à-dire un résultat moins bon que celui de la simple moyenne. Il faut cependant retenir que tous les modèles sous-estiment les délais extrêmes.

### Question du guide : Le GNN apporte-t-il un gain réel ?

Oui, par rapport à la référence linéaire et par rapport au perceptron multicouche sans graphe : en protocole « random », l'erreur quadratique moyenne diminue de 36 % et de 37 % respectivement. En revanche, le GNN n'apporte pas de gain par rapport à HistGradientBoosting lorsque celui-ci dispose de caractéristiques de charge

(3,15 contre 2,12), et il n'en apporte pas non plus sous changement de distribution (6,38 contre 3,37, soit un résultat pire que celui de Ridge).

**Question du guide : Quelles caractéristiques comptent le plus ?**

Les caractéristiques décisives sont la charge du goulot d'étranglement (ρ maximal sur le chemin), le rapport entre le débit garanti par WFQ et le débit demandé, la charge de la tranche eMBB, le débit de paquets et la part WFQ minimale. Le type de tranche et la structure du chemin sont secondaires dès que la charge est connue.

**Question du guide : Que se passe-t-il quand la distribution change ?**

Les performances se dégradent : l'erreur quadratique moyenne globale est multipliée par 1,6 pour HistGradientBoosting et par 2,0 pour le GNN, et elle est multipliée par 1,7 sur les événements pour le GNN. Cette dégradation est liée à la fois à la réduction des données d'entraînement et à l'apparition de topologies et de contentions hors du domaine appris.

> **« Meilleur score » ≠ « meilleure solution ».** HistGradientBoosting obtient les meilleurs scores, mais il dépend de caractéristiques de charge choisies à la main grâce à une connaissance du simulateur (routage, WFQ). Le GNN obtient environ 60 % de ce gain en n'utilisant que le graphe brut : il est plus générique, car il ne demande aucune ingénierie de caractéristiques, mais il est moins précis et moins robuste dans notre étude. Le choix dépend donc du contexte d'usage (section 11).

# 10. Limites de notre approche

Nous présentons ici les limites de notre étude. Pour chacune, nous indiquons ce qu'elle implique pour l'interprétation des résultats et la piste que nous proposerions pour la lever.

***Tableau 22** – Limites de l'approche, conséquences et pistes*

| Limite | Conséquence sur l'interprétation | Piste pour la lever |
|---|---|---|
| **Pas de dynamique temporelle** : simulations indépendantes, une seule intensité de trafic. | Nous prédisons un régime stationnaire par flux, pas l'évolution dans le temps ; l'« événement » est un basculement de régime entre flux et non une variation temporelle réelle. | Disposer de séries temporelles de trafic et de qualité de service par flux. |
| **Peu de simulations indépendantes** (165, dont 108 pour l'entraînement). | Le GNN, qui apprend sur des graphes entiers, est limité par ce volume ; les 37 312 flux sont fortement corrélés au sein d'une simulation. | Générer davantage de simulations et de tailles ; augmentation de données par sous-graphes. |
| **Événements peu nombreux et concentrés** (6,8 %, certaines simulations avec plus de 90 % d'événements). | Les métriques sur les événements reposent sur peu de simulations ; l'événement étant défini à partir de la cible, l'analyse est a posteriori, et les $R^2$ sur ce sous-ensemble sont négatifs par construction (faible variance interne). | Privilégier la MSE et la MAE sur les événements ; pondérer la perte en faveur des événements. |
| **Un seul type de changement testé** : la taille du réseau, confondue avec la quantité de données d'entraînement. | D'autres changements (intensité, routage, politique WFQ) ne peuvent pas être testés avec ce jeu ; l'effet « taille » et l'effet « moins de données » ne sont pas séparés. | Construire des protocoles de test sur d'autres axes et comparer à quantité de données égale. |
| **Validation de même distribution que l'entraînement.** | Elle ne détecte pas le changement de distribution (le GNN a une MSE de validation de 1,67 pour un test à 6,5). | Utiliser une validation « shiftée » ou une validation par groupes de taille. |
| **Variabilité entre graines** non négligeable pour le GNN (0,24 en « random », 0,71 en « shift »). | Les ablations et la comparaison d'architectures ont été faites avec une seule graine : les écarts inférieurs à 0,3 en « random » ne sont pas significatifs. | Répéter les ablations sur plusieurs graines et donner des intervalles. |
| **Prédiction et non décision** : le modèle prédit la qualité de service d'une allocation donnée. | L'optimisation de l'allocation (choix des poids WFQ et des chemins) n'est pas réalisée. | Boucle de décision : recherche sur des allocations candidates ou apprentissage par renforcement. |
| **Caractéristiques planifiées incohérentes** avec le routage simulé. | Elles peuvent induire le modèle en erreur (leur retrait améliore le résultat en « shift »). | Les exclure ou les recalculer à partir du routage simulé. |

# 11. Réponse à la question finale et conclusion

**Question finale.** « Dans quelles conditions une approche GNN apporte-t-elle une valeur ajoutée pour la gestion dynamique des slices B5G ? »

## 11.1 Les conditions dans lesquelles le GNN apporte une valeur ajoutée

1. **Quand la contention n'est pas décrite par des caractéristiques ingéniérées.** À caractéristiques individuelles identiques, le graphe réduit la MSE de 37 % par rapport à un perceptron multicouche et de 36 % par rapport à Ridge, car il agrège la charge par les files partagées. Il retrouve environ 60 % du gain obtenu avec la charge calculée à la main, sans connaître le routage ni la formule de ρ.
2. **Quand le test suit la distribution d'entraînement** (même gamme de topologies). En protocole « random », le GNN est fiable ($R^2$ d'environ 0,41), mais il reste inférieur à HistGradientBoosting.
3. **Quand l'architecture respecte la physique du problème.** L'agrégation par moyenne normalisée (GCN) échoue (MSE de 4,61) ; les agrégations qui conservent le goulot d'étranglement (GraphSAGE avec moyenne et maximum, attention GATv2) fonctionnent. Le choix d'architecture doit être relié au phénomène physique, pas à la popularité de la méthode.
4. **Quand un opérateur ne peut pas maintenir de caractéristiques de charge manuelles** (topologies variables, services nouveaux). Le graphe offre alors une représentation générique ; hybridé avec des caractéristiques de charge, il atteint le niveau de HistGradientBoosting (2,116 contre 2,119).

## 11.2 Les conditions dans lesquelles il n'en apporte pas

- **Face à un modèle classique doté de caractéristiques de congestion pertinentes :** ce dernier est plus simple, plus rapide et plus précis.
- **Lors d'un changement brusque de taille de réseau sans réentraînement :** le GNN se dégrade d'un facteur 2 et devient pire que la référence linéaire, alors que HistGradientBoosting reste utilisable.
- **Quand les données sont rares :** avec 165 simulations, le GNN manque d'exemples de graphes pour généraliser.

## 11.3 Réponse à la question centrale

**Réponse.** Oui, on peut apprendre à prédire la qualité de service (le délai) d'un flux à partir de la configuration du réseau, avec une erreur nettement meilleure que celle de la référence (MSE de 2,12 contre 4,93 ; $R^2$ d'environ 0,59). La fiabilité sous changement brusque est **partielle** : un modèle à caractéristiques physiques (HistGradientBoosting) reste utile (MSE de 3,37), tandis que le GNN nécessite un réentraînement ou des données plus variées. Les événements (saturation, effondrement) restent le point faible de tous les modèles : ils sont sous-estimés, et leur détection fiable passe davantage par l'état de charge du goulot d'étranglement que par un modèle de régression.

## 11.4 Trois choix méthodologiques défendus (préparation de la soutenance L7)

*Tableau 23 – Les trois choix méthodologiques défendus*

| Choix méthodologique | Argument | Alternative écartée |
|---|---|---|
| **1. Séparation par simulation et non par flux, avec un protocole « shift »** | Les flux d'une simulation partagent les mêmes liens et les mêmes files : une séparation par flux ferait fuir l'information de contention vers le test. Le protocole « shift » permet de tester la robustesse au changement brusque. | Séparation aléatoire par flux, qui aurait donné des scores artificiellement optimistes. |

| Choix méthodologique | Argument | Alternative écartée |
|---|---|---|
| **2. Cible ln(délai) et caractéristiques issues des entrées et du routage réel** | Le logarithme gère six ordres de grandeur ; utiliser le routage simulé (corrélation 0,93–0,98 avec l'utilisation réelle) et non le plan (0,4–0,7), et exclure toute sortie du simulateur, évite les fuites d'information. | Prédire le délai brut ; utiliser les chemins planifiés ou l'utilisation simulée des liens. |
| **3. Événement par score z robuste par tranche (avec effondrement), et graphe hétérogène avec contrôle « MLP sans arêtes »** | Le seuil par tranche évite de confondre URLLC rapide et eMBB lent, et la vérification visuelle (simulation 115) valide le masque. Le contrôle sans arêtes isole l'apport réel du graphe. | Seuil global sur le délai ; comparer le GNN uniquement à Ridge. |

## 11.5 Perspectives

- Entraîner sur davantage de simulations et de tailles de réseaux, avec une augmentation de données par sous-graphes.
- Valider sur des groupes de tailles de réseaux, afin que la sélection de modèle détecte le changement de distribution.
- Tester des GNN hybrides combinant caractéristiques de charge et agrégation, qui ont déjà atteint le niveau de HistGradientBoosting en « random ».
- Ajouter une perte pondérée en faveur des événements, pour réduire la sous-estimation des délais extrêmes.
- Utiliser le prédicteur dans une boucle d'optimisation de l'allocation (poids WFQ et routage), ce qui transformerait notre modèle de prédiction en véritable outil de décision.
- Vérifier et relancer l'ablation du groupe « trafic » (section 9.3), puis répéter les ablations du GNN sur plusieurs graines.

## 11.6 Conclusion générale

Ce projet nous a permis de parcourir toute la chaîne d'une étude d'apprentissage appliquée à la gestion des tranches de réseau : exploration des données et de leurs pièges, formulation d'un problème de prédiction de la qualité de service, définition d'événements physiquement vérifiés, construction d'un graphe hétérogène et comparaison rigoureuse d'un modèle de référence, d'un modèle d'arbres et d'un réseau de neurones sur graphes. Notre principal enseignement est que la **congestion du goulot d'étranglement** est l'information centrale, et que la valeur d'un GNN ne se mesure pas au seul score : elle dépend de la disponibilité de caractéristiques de congestion, de la quantité de données et de la stabilité de la distribution du réseau. Un meilleur score ne signifie pas une meilleure solution : HistGradientBoosting est le plus précis ici, tandis que le GNN offre une représentation plus générique, prometteuse dès que les données seront plus nombreuses et plus variées.

# Annexe — Reproductibilité des résultats

Les commandes ci-dessous permettent de recréer l'environnement et de régénérer l'ensemble des tableaux et des figures. L'exécution complète dure environ 40 à 60 minutes, le GNN étant l'étape la plus longue.

```
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
cd src
python run_all.py            # environ 40 à 60 min (GNN : étape la plus longue)
python run_all.py --quick    # test rapide : GNN 8 époques, 1 graine
# ou étape par étape :
# 01_extract.py → 02_eda.py → 03_events.py → 04_baseline.py → 05_gnn.py → 06_compare.py
```

Les résultats sont écrits dans `results/tables/*.csv` (tableaux L1 à L6) et `results/figures/*.png`. Les valeurs de ce rapport proviennent de ces fichiers ; certaines figures de synthèse (démarche, protocoles, graphe, architectures, comparaisons et ablations) ont été produites à partir de ces mêmes tableaux.

### Extrait : définition des événements (features.py)

```
z = (ln_delay - median_slice) / (1.4826 * MAD_slice)
event = (z > 3.0) | (loss_ratio > 0.90)
```

### Extrait : passage de messages (05_gnn.py)

```
x = sum(encoder[type](features_type))        # un encodeur par type de nœud
for conv, norm in zip(convs, norms):             # SAGE(mean,max) / GCN / GATv2
    x = x + dropout(relu(norm(conv(x, edge_index))))
y_hat = head(x)[flow_nodes]                      # ln(délai) des flux
```